AI 500
Foundations of Artificial Intelligence

L2: Search Algorithms

By Murtaza Taj

LUMS
A Not-for-Profit University

Playing game measure intelligence

AI to defeat world chess champion

Example: Route Finding Problem

Goal: Shortest route from Arad to Bucharest

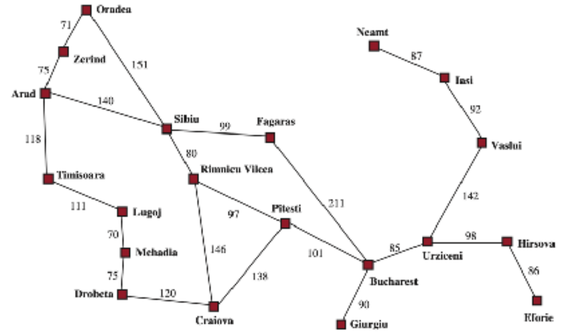
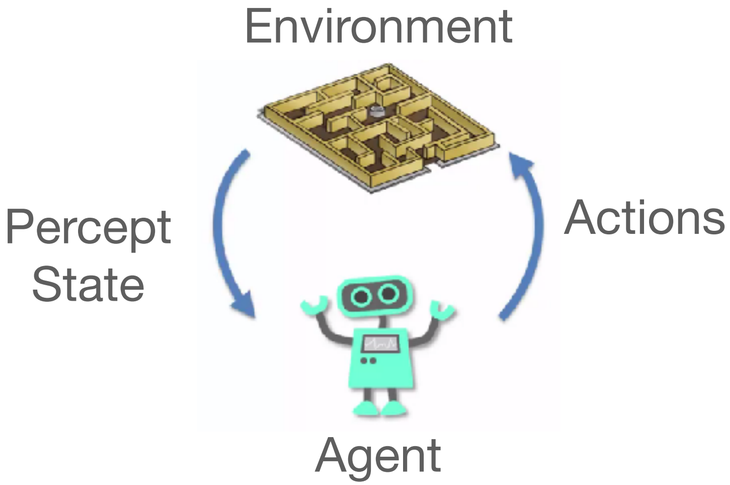


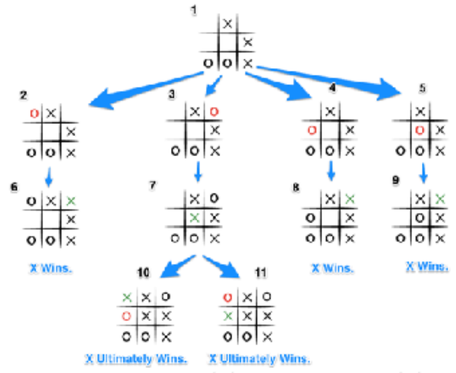
Figure 3.1 A simplified road map of part of Romania, with road distances in miles.

Problem Solving Agent

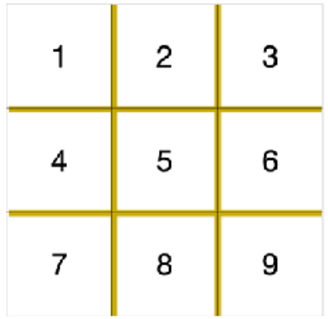


State / Action Space

- **State Space**
  - Number of possible configurations of the environment for a given problem
- **Action Space**
  - Number of actions possible from a state
  - Given as average worst-case
- Used as an estiamtion of complexity

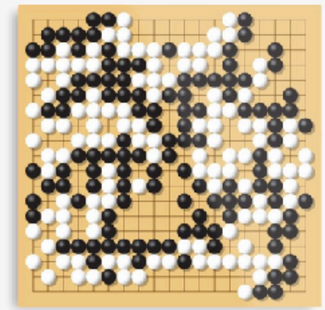


State-Space Complexity Calculation



10^4

10^{42}



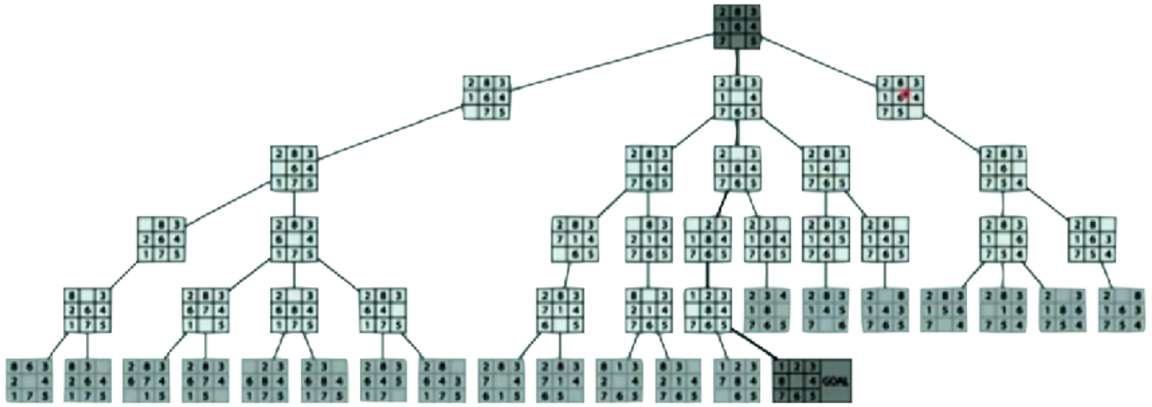
10^{172}

Search Algorithms

8 Puzzle: Search Tree

8		6
5	4	7
2	3	1

	1	2
3	4	5
6	7	8



Tic-Tac-Toe

Search Tree

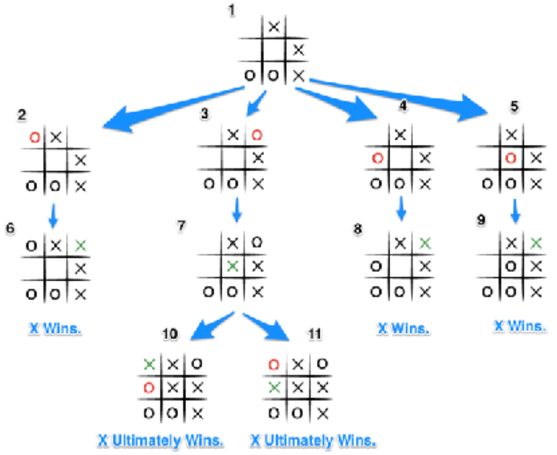
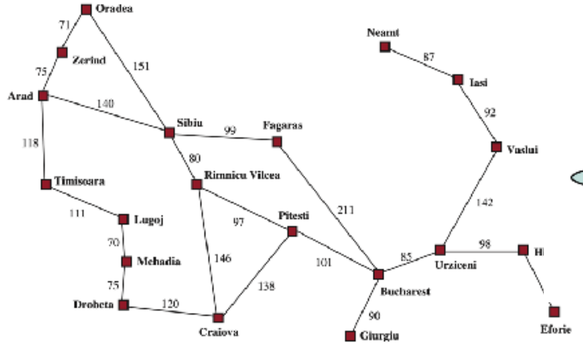


Image credits: https://www.neverstopbuilding.com/blog/minimax

Example: Route Finding Problem

Goal: Shortest route from Arad to Bucharest



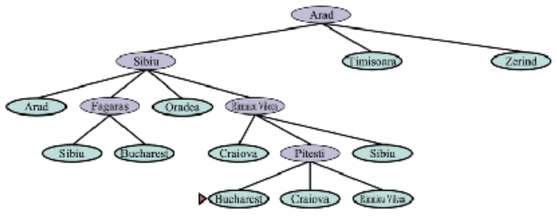
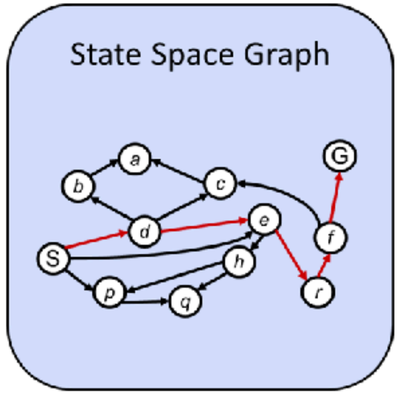


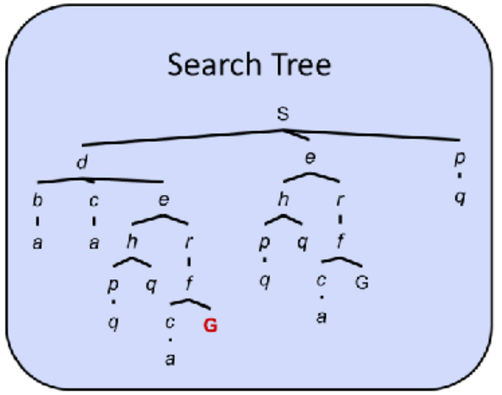
Figure 3.1 A simplified road map of part of Romania, with road distances in miles.

Search Algorithms



Each NODE in in the search tree is an entire PATH in the state space graph.

We construct both on demand – and we construct as little as possible.



A Generic Search Algorithm:

- Given a graph, start nodes, and a goal description, incrementally explore paths from the start nodes.
- Maintain a *frontier/Queue* of nodes that are to be explored.
- As search proceeds, the frontier expands into the unexplored nodes until a goal node is encountered.
- The way in which the **frontier** is **expanded** defines the **search strategy**.

Generic Tree Search

***Tree search*: Don't check if nodes are visited multiple times**

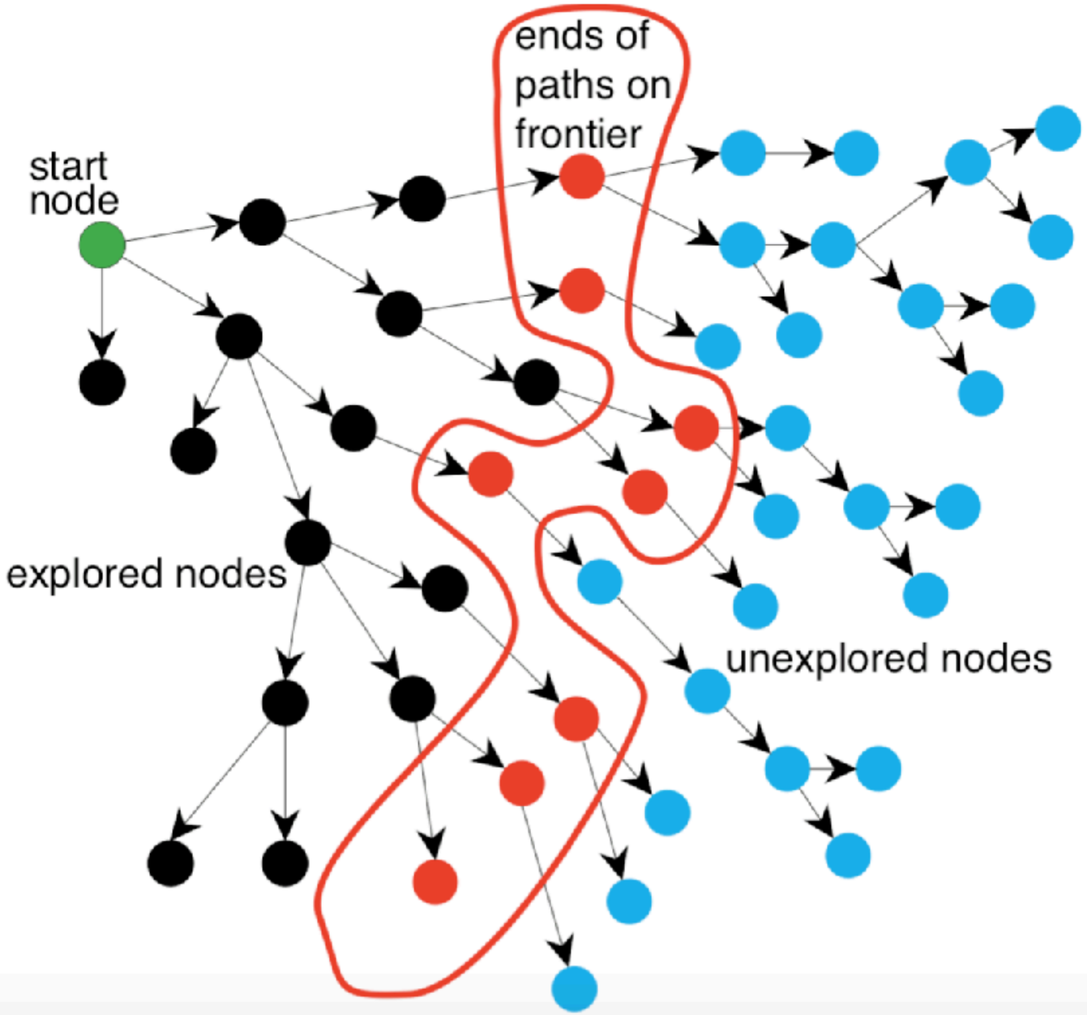
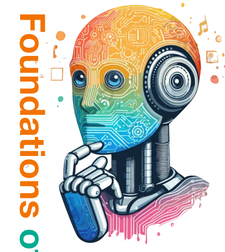


Image Credits: Peter Ljunglöf

Turning Tree Search Into Graph Search

***Graph search*: Keep track of visited nodes**




```
function Search(graph, initialState, goalState):
    initialise frontier using the initialState
    initialise exploredSet to the empty set
    while frontier is not empty:
        select and remove node from frontier
        if node.state is a goalState then return Solution
        add node to exploredSet
        for each child in ExpandChildNodes(node, graph):
            if child is not in frontier or exploredSet
                add child to frontier
    return failure
```

Search Algorithms

Uniformed (Blind)

- Breadth First Search (BFS)
- Depth First Search (DFS)
- Uniform Cost Search (UCS)
- Depth Limited Search (DLS)
- Iterative Deepening Depth First Search (IDDFS)

(Russel & Norvig 3.4)

Informed (Heuristic)

- Best First Search
  - Greedy Best First SearchSearch
  - A*
  - Recursive Best First Search (RBFS)
- Beam Search

(Russel & Norvig 3.5)

Uninformed Search

Breadth First Search (BFS)

Depth First Search (DFS)

Uniform Cost Search (UCS)

Depth Limited Search (DLS)

Iterative Deepening Depth First Search (IDDFS)

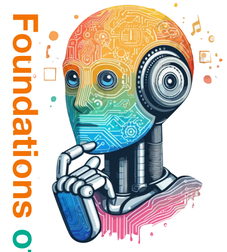
(Russel & Norvig 3.4)

Breadth-First Search (BFS)

Breadth-First Search

- Root node expanded first
- Root successor expanded next
- Their successor expanded next ….
- BFS in General
  - All node at a **given depth** are expanded before any node in the next level

Breadth-First Search



- *Breadth-first search* treats the frontier as a queue.
- It always selects one of the earliest elements added to the frontier.
- If the list of paths on the frontier is $[p_1, p_2, \cdots, p_r]$, then:
  - p_1 is selected (and removed).
  - Its neighbors are added to the end of the queue, after p_r
  - p_2 is selected next.

Illustrative Graph: Breadth-First Search

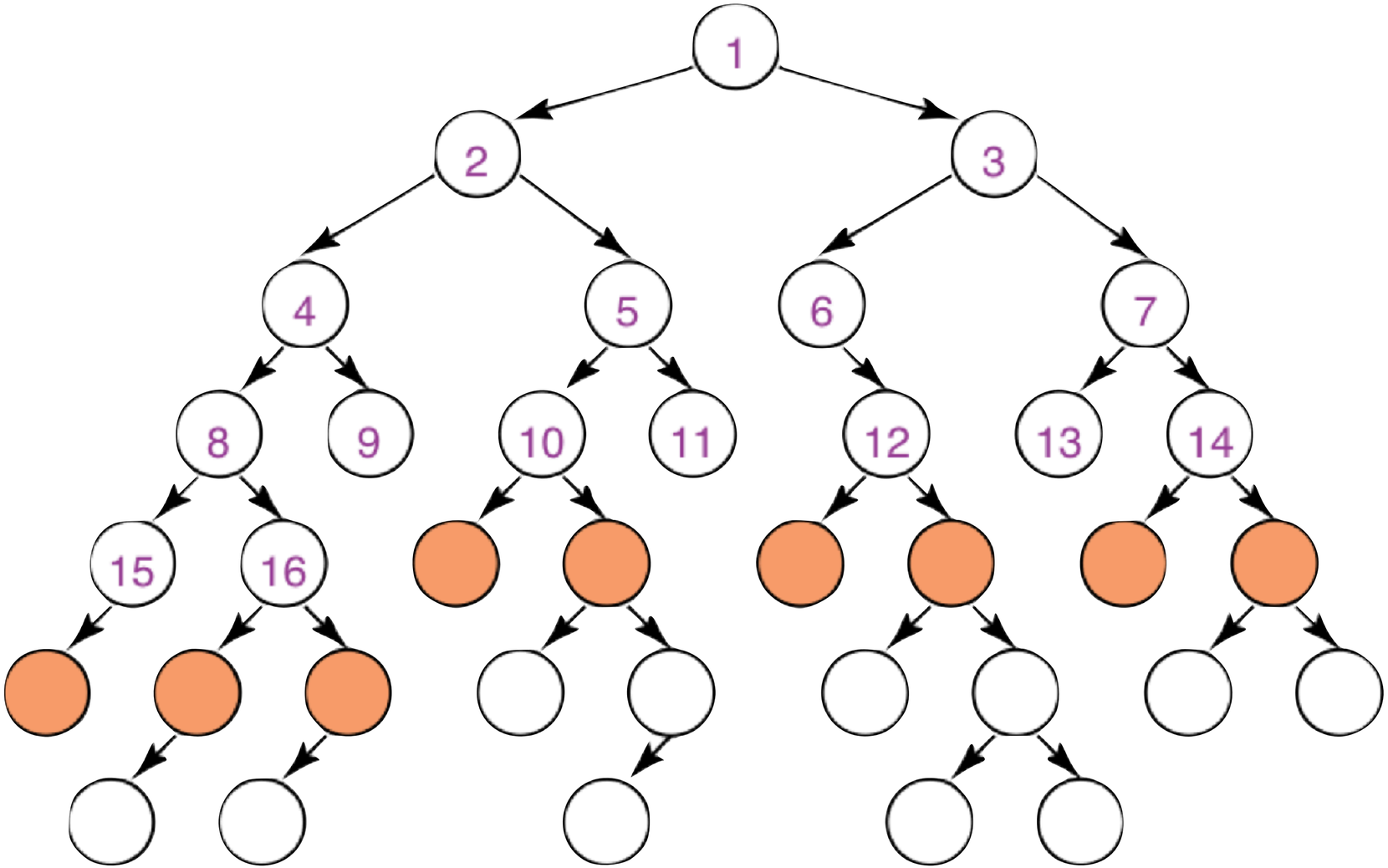
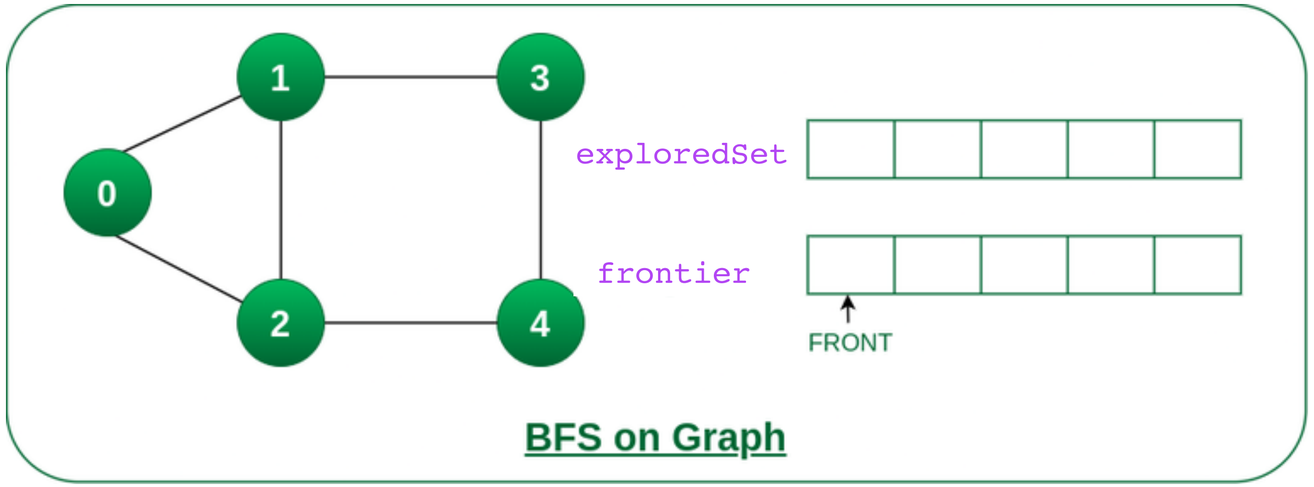


Image Credits: Peter Ljunglöf

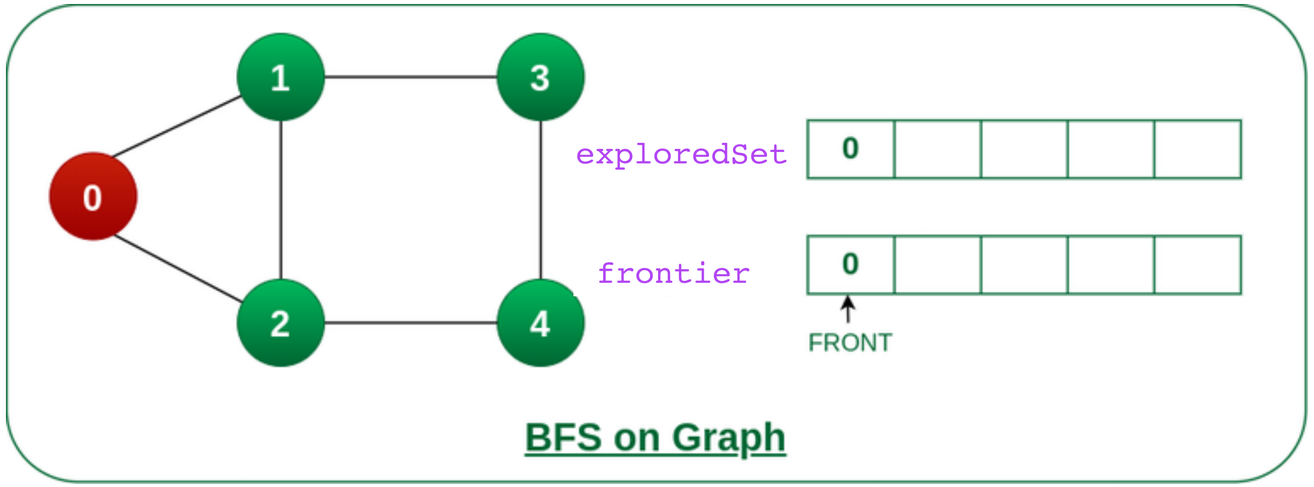
Breadth First Search

Step1: Initially frontier and exploredSet arrays are empty.



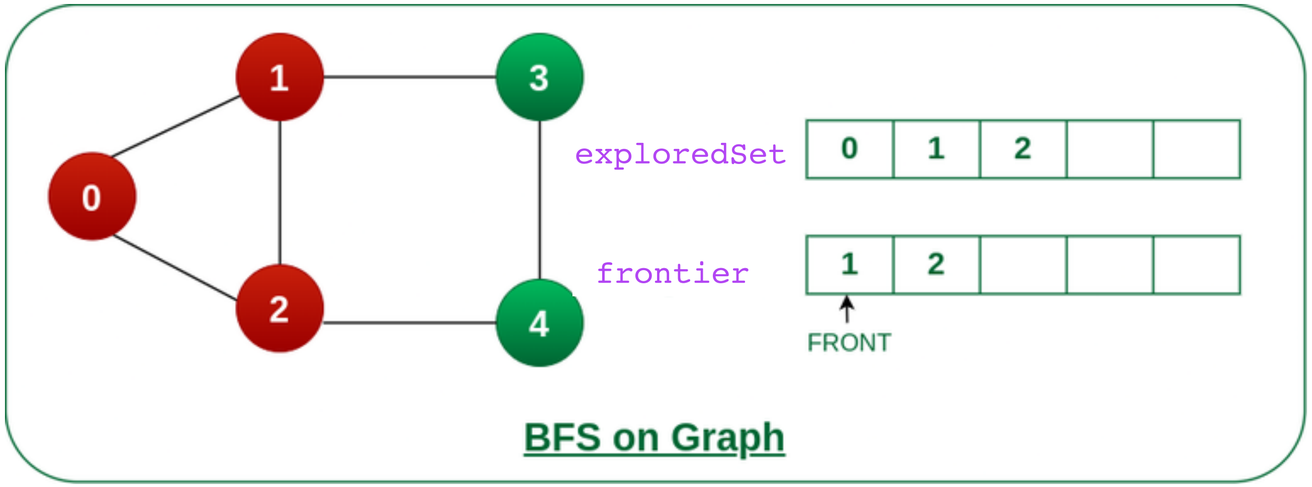
Breadth First Search

Step2: Push 0 into queue and mark it visited



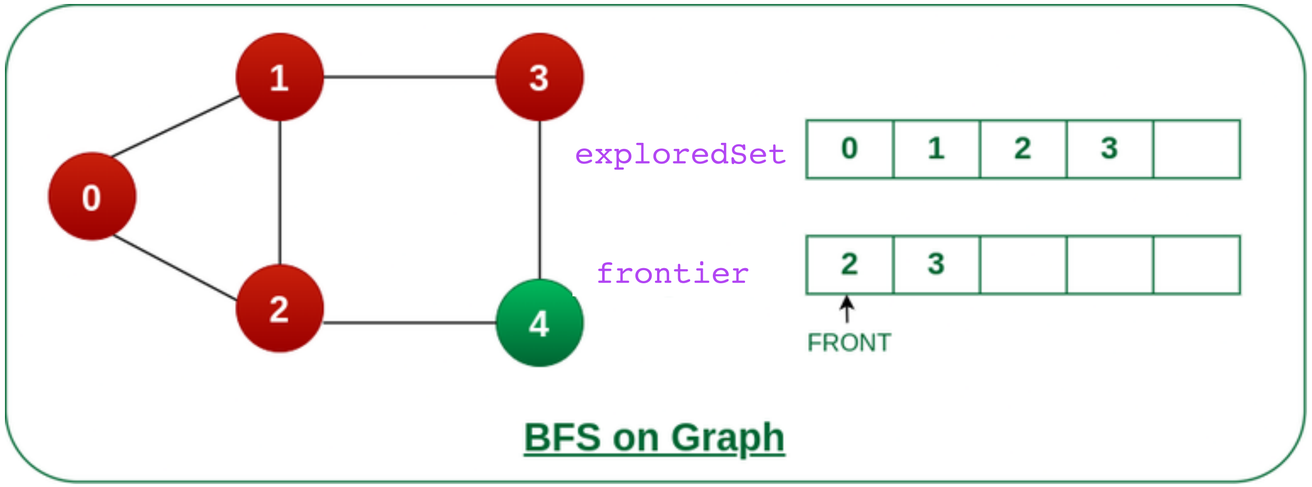
Breadth First Search

Step 3: Remove 0 from the front of queue and visit the unvisited neighbours and push them into queue.



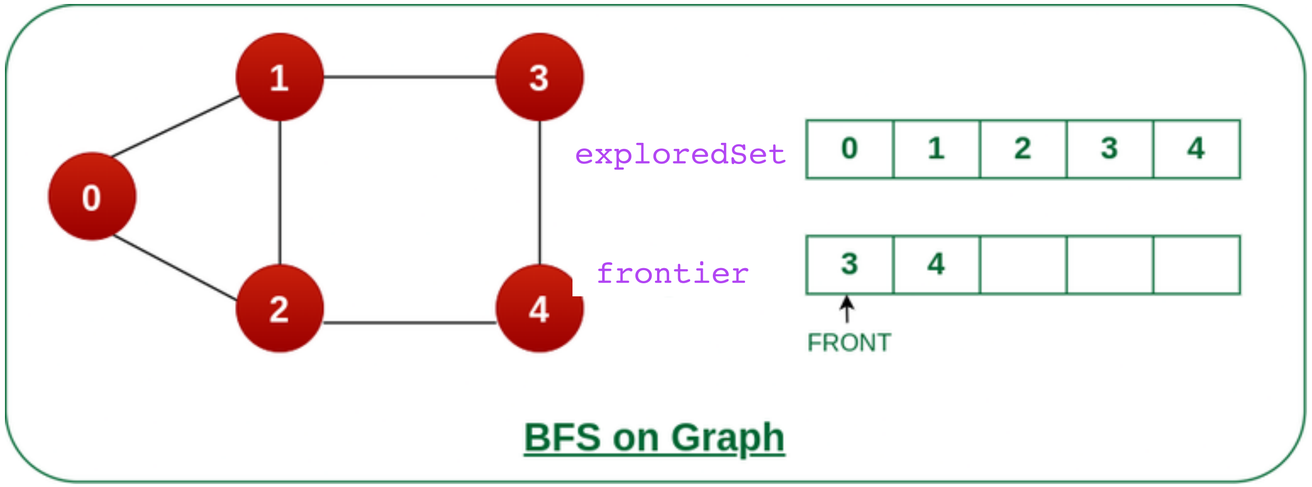
Breadth First Search

Step 4: Remove node 1 from the front of queue and visit the unvisited neighbours and push them into queue.



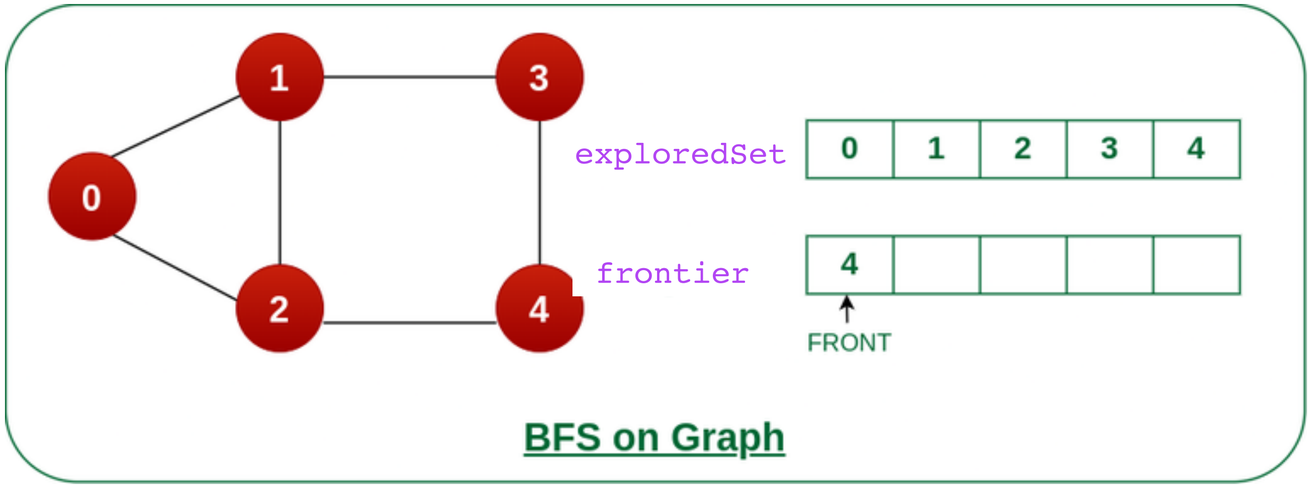
Breadth First Search

Step 5: Remove node 2 from the front of queue and visit the unvisited neighbours and push them into queue.



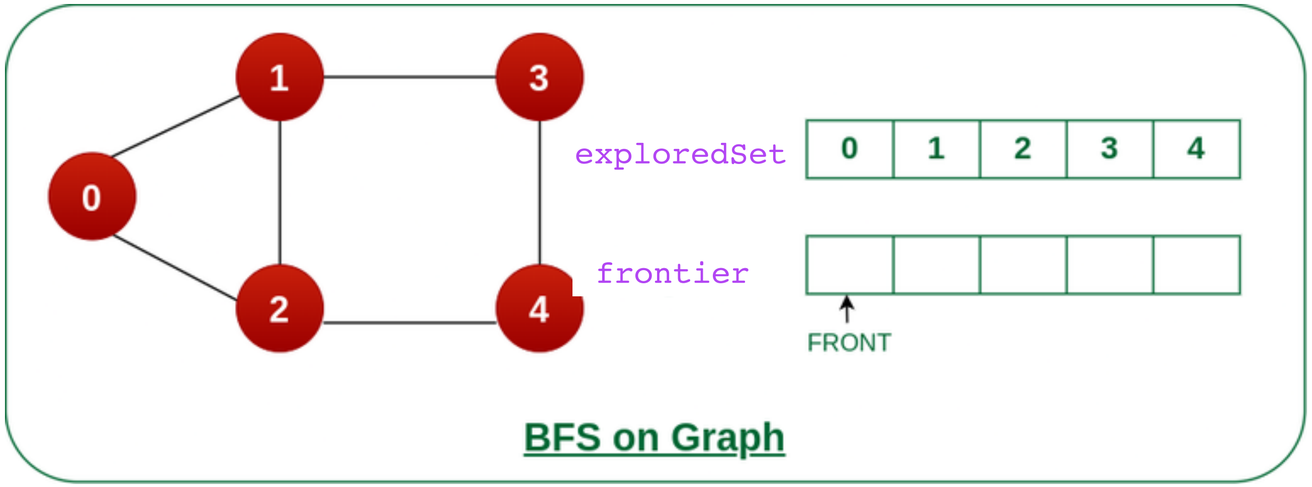
Breadth First Search

Step 6: Remove node 3 from the front of queue and visit the unvisited neighbours and push them into queue.



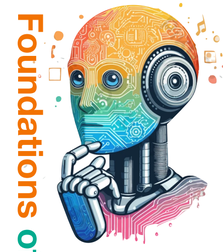
Breadth First Search

Steps 7: Remove node 4 from the front of frontier and visit the unvisited neighbours and push them into frontier.



Breadth First Search

Examples



Breadth-First Search

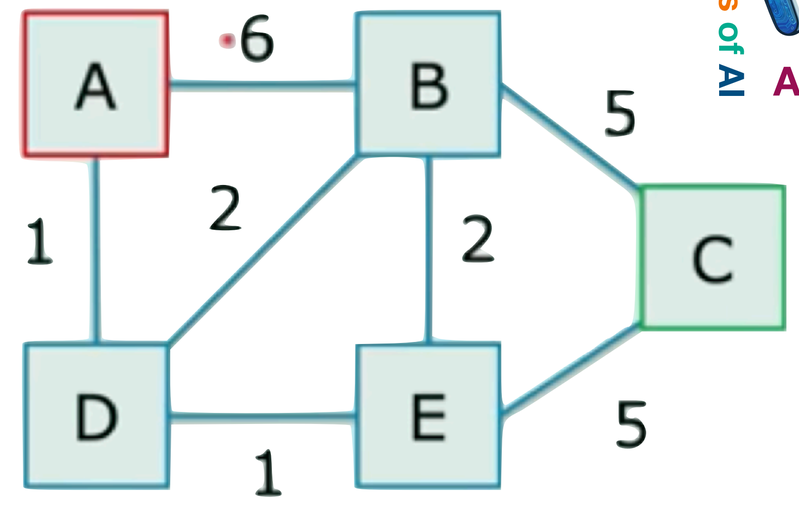
Pseudo code

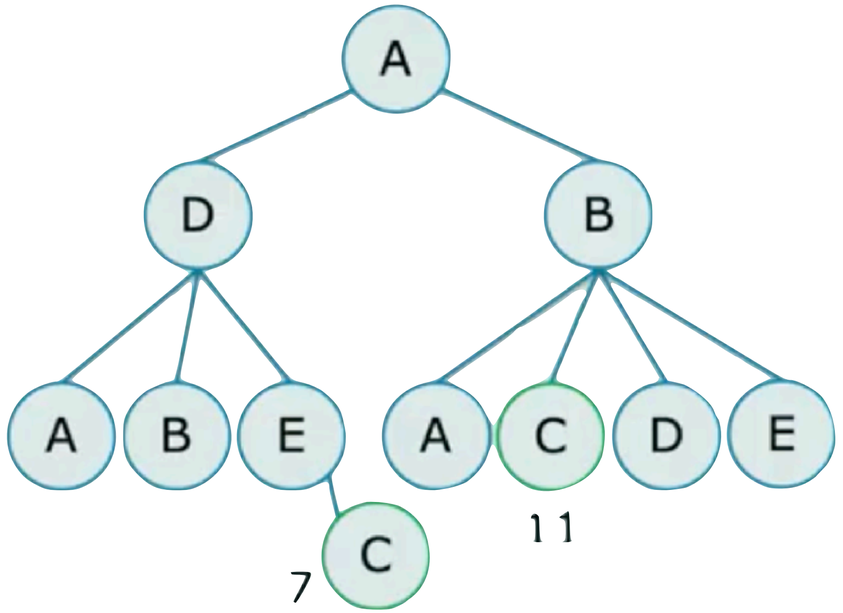
```
function Search(graph, initialState, goalState):
    initialise frontier as Queue using the initialState
    initialise exploredSet to the empty set
    while frontier is not empty:
        select and remove node from frontier
        if node.state is a goalState then return solution
        add node to exploredSet
        for each child in ExpandChildNodes(node, graph):
            if child is not in frontier or exploredSet
                add child to frontier
    return failure
```

Breadth First Search

Problem Solving Performance

- **BFS is Complete**
  - BFS explores entire tree, so it will find a goal node
  - BFS will find the shallowest goal node, since it explores all nodes at a given depth before the nest depth
- **BFS is not Optimal**
  - Shallowest node is not necessarily optimal
  - BFS is optimal if path cost is non-decreasing function on depth
  - BFS is optimal if action cost are all the same

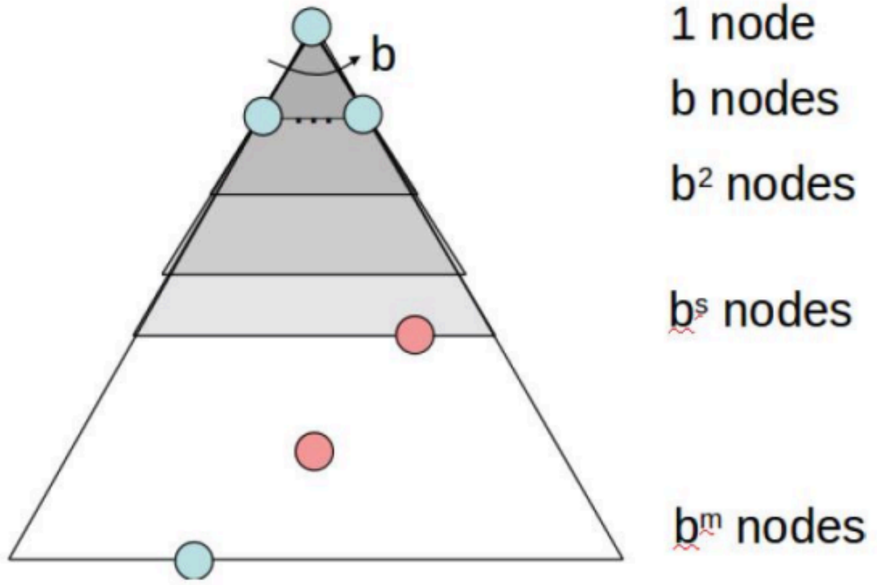




Breadth First Search

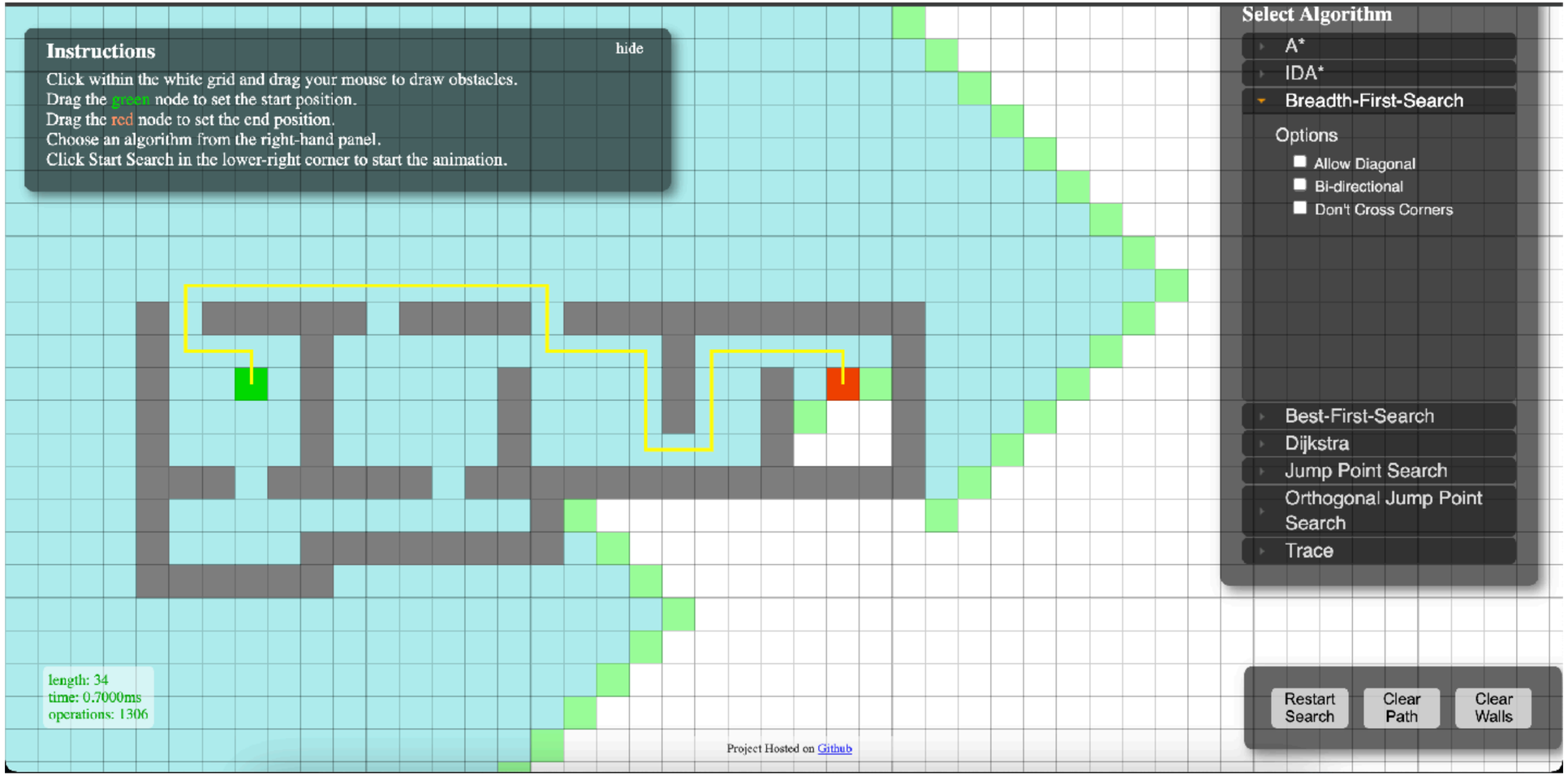
Problem Solving Performance

- **Time Complexity** $O(b^d)$
  - Each state has b children
  - Consider goal at depth d
  - $b \times b \times b \cdots = O(b^d)$ nodes generated
- **Space Complexity** $O(b^d)$
  - Number of nodes in memory = size of frontier



BFS - Demo

https://qiao.github.io/PathFinding.js/visual/



Uniform Cost Search (UCS)

Uniform Cost Search (UCS)

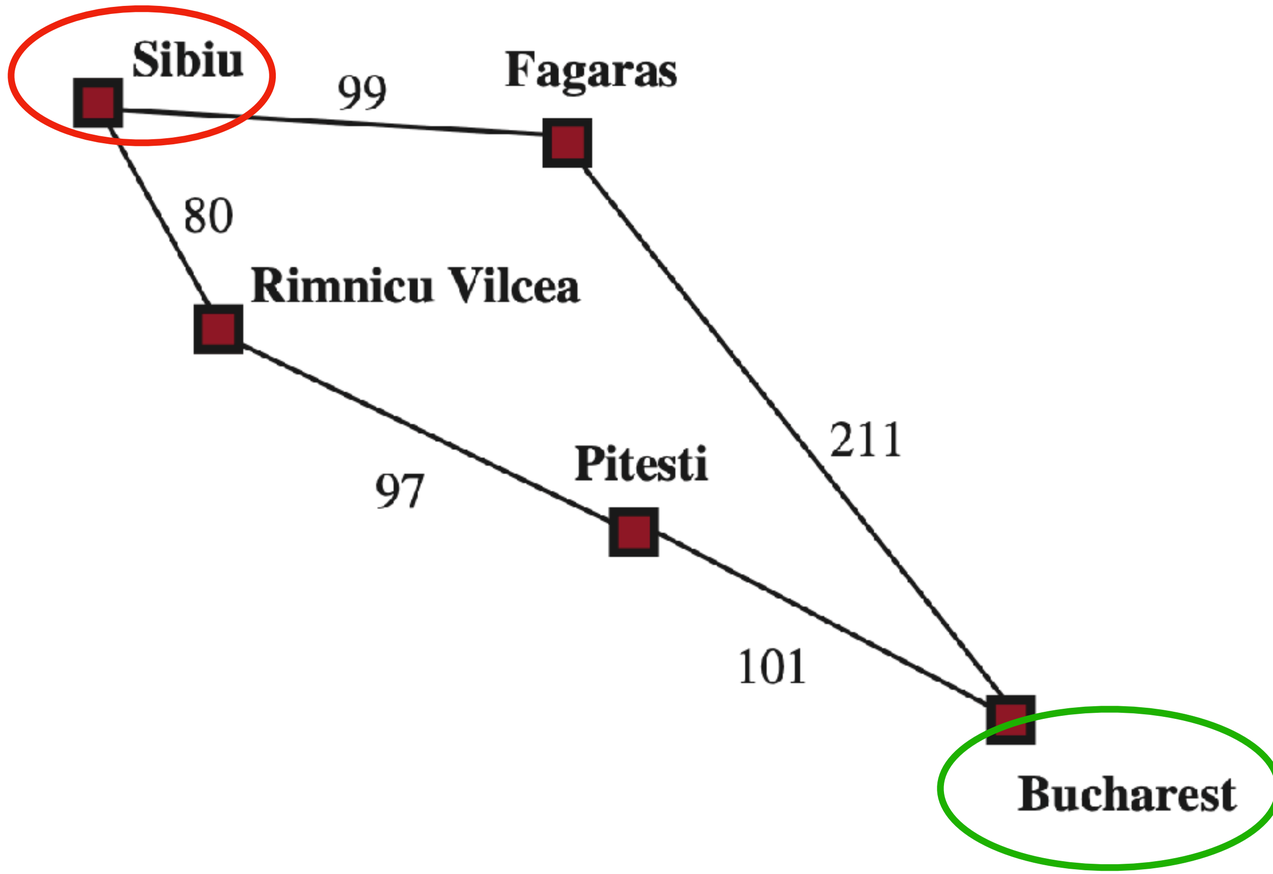
- BFS optimal when all action cost equal because it expands the *shallowest* node
- UCS expands node with *lowest path cost*
- If all costs are equal, equivalent to BFS
- Works only if all costs > 0
- AKA Dijkstra's algorithm

.

Example: UCS

Step 1:

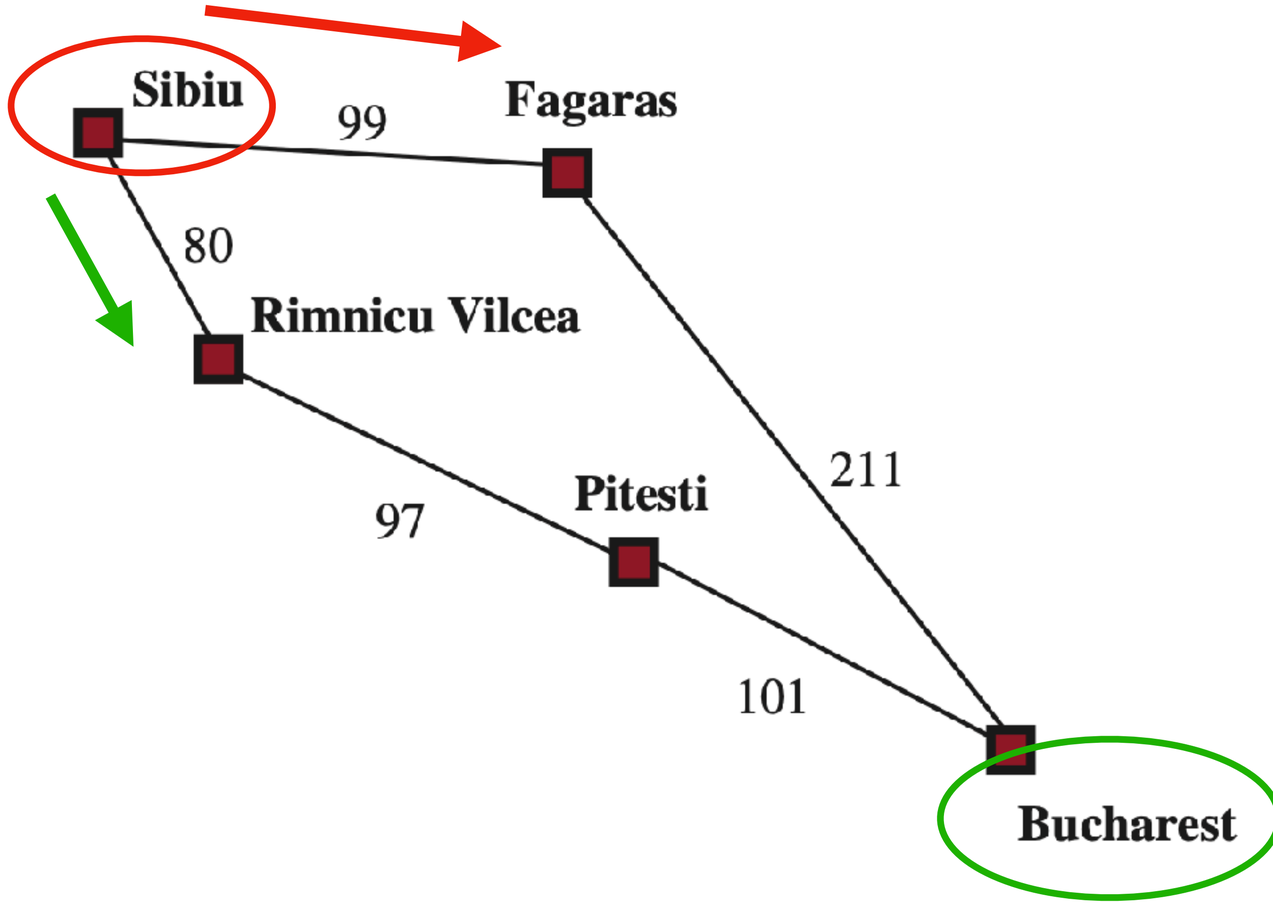
- Visited: Sibiu
- Frontier: Rimnicu Vilcea, Fargaras



Example: UCS

Step 2:

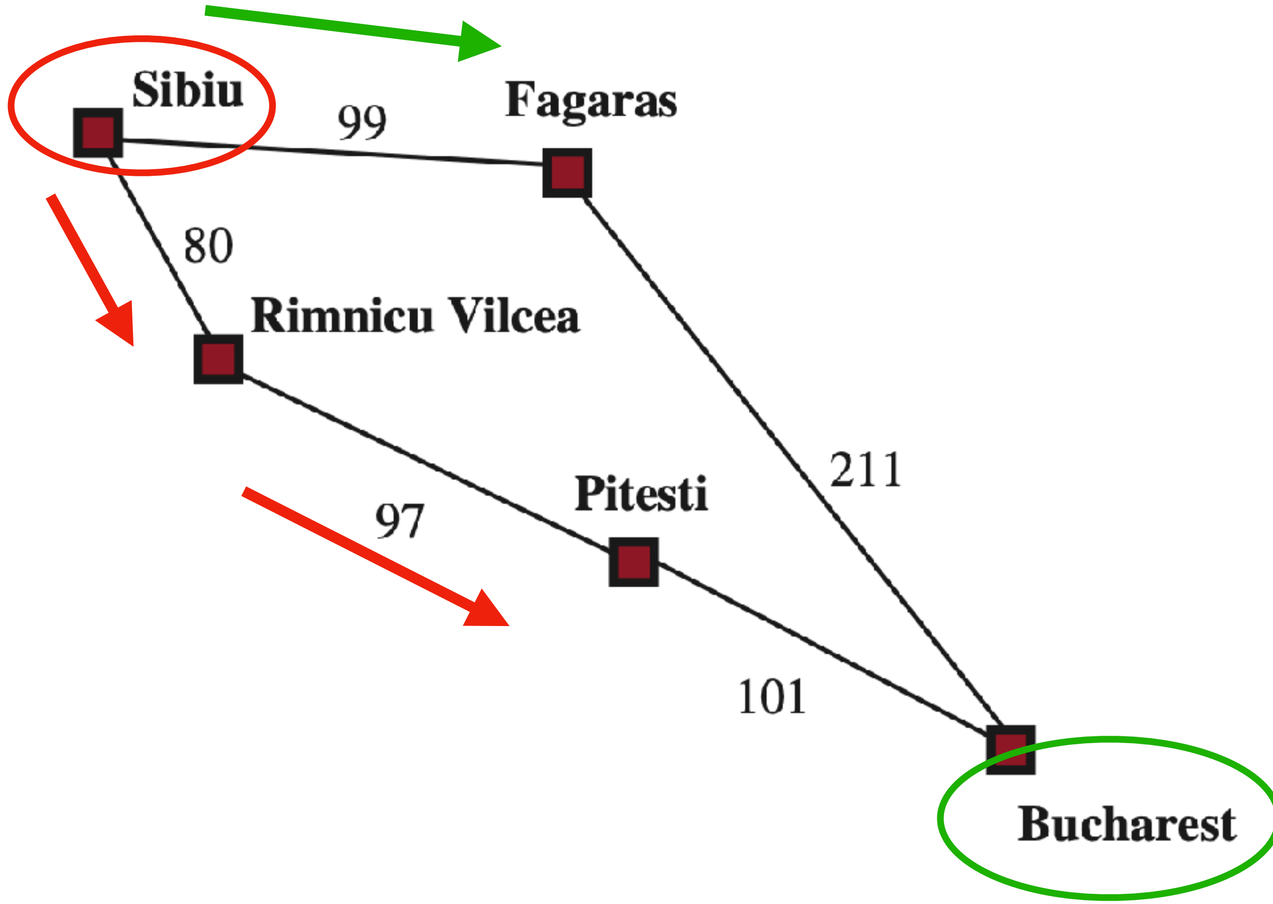
- Visited: Sibiu, Rimnicu Vilcea
- Frontier: Fargaras, Pitesti
- Cost: Sibiu -> Rimnicu Vilcea: 80



Example: UCS

Step 3:

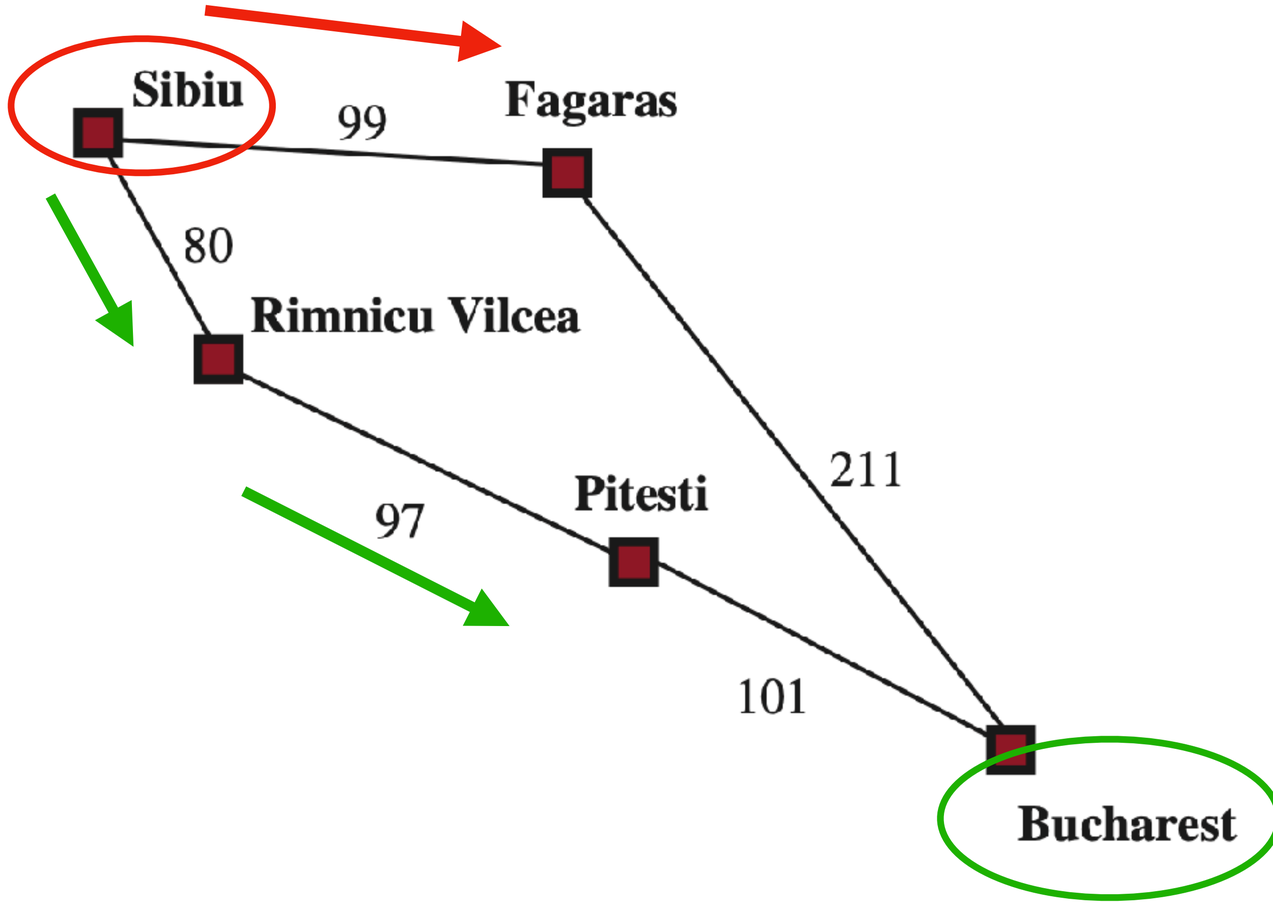
- Visited: Sibiu, Rimnicu Vilcea, Fargaras
- Frontier: Pitesti, Bucharest
- Cost: Sibiu -> Fargaras: 99



Example: UCS

Step 4:

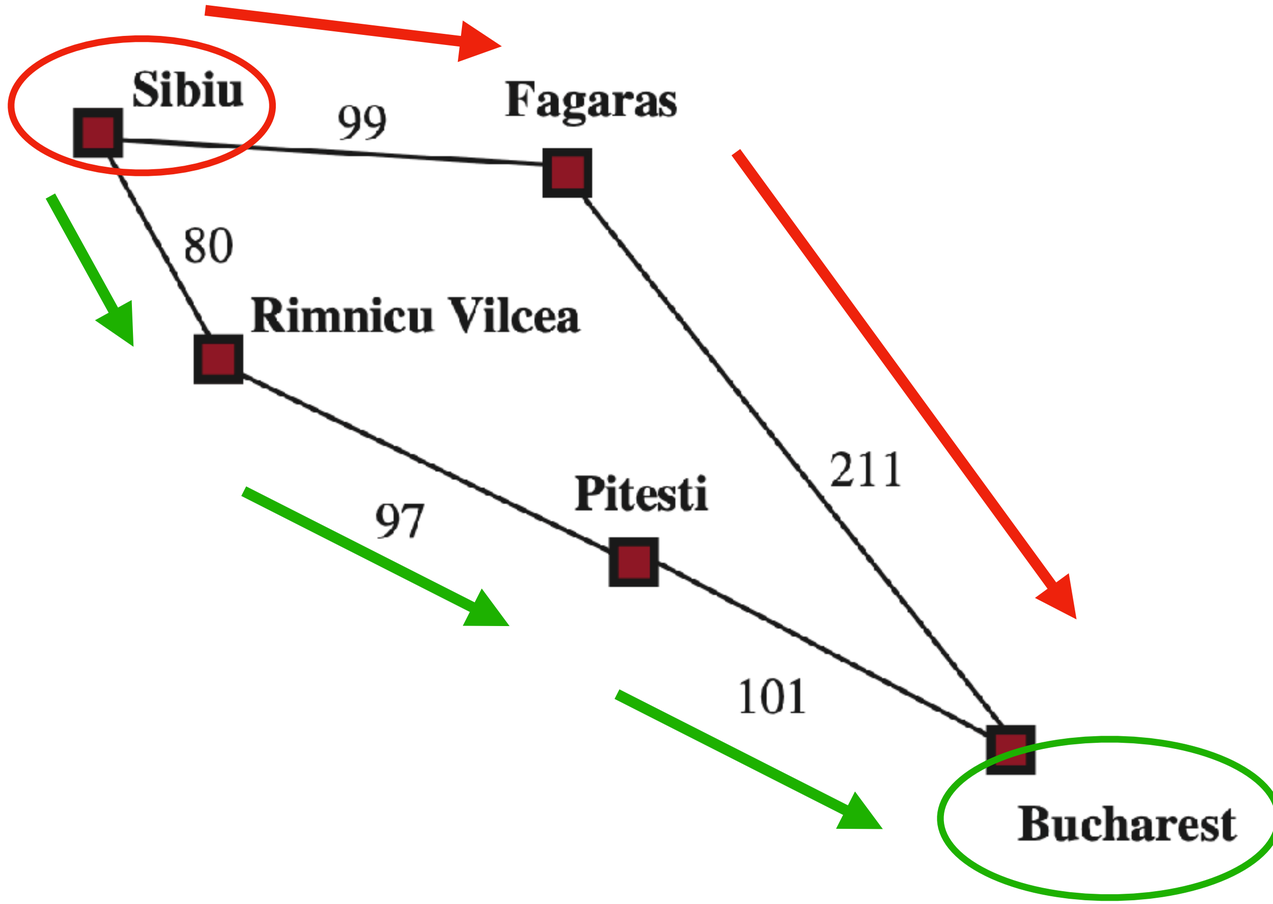
- Visited: Sibiu, Rimnicu Vilcea, Pitesti
- Frontier: Bucharest
- Cost: Sibiu -> Rimnicu Vilcea -> Pitesti: 177



Example: UCS

Step 5:

- Visited: Sibiu, Rimnicu Vilcea, Fagaras, Pitesti
- Frontier: x
- Cost: Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest: 80+97+101 = 278



Uniform-Cost Search

Pseudo code

```
function Search(graph, initialState, goalState):
    initialise frontier using the initialState
    initialise exploredSet to the empty set
    while frontier is not empty:
        select and remove node with minimum path cost from frontier
        if node.state is a goalState then return node
        add node to exploredSet
        for each child in ExpandChildNodes(node, graph):
            if child is not in frontier or exploredSet
                add child to frontier
    return failure
```

Uniform Cost Search

Problem Solving Performance

- UCS is **complete** if b is finite
- UCS is **optimal** if action cost > 0 (ϵ)
- **Time Complexity**
  - Measured by path costs, not depth
  - Can't use b and d for complexity
  - Let C^* be the cost of optimal solution
  - Complexity = $O(b^{1+C^*/\epsilon})$
  - Worst case this can be much bigger than b^d

Depth-First Search (DFS)

Depth-First Search

- *Depth-first search* treats the frontier as a stack.
- It always selects one of the last elements added to the frontier.
- If the list of nodes on the frontier is $[p_1, p_2, p_2, \cdots]$, then:
  - p_1 is selected (and removed).
  - Nodes that extend p_1 are added to the front of the stack (in front of p_2).
  - p_2 is only selected when all nodes from p_1 have been explored.

Illustrative Graph: Depth-First Search

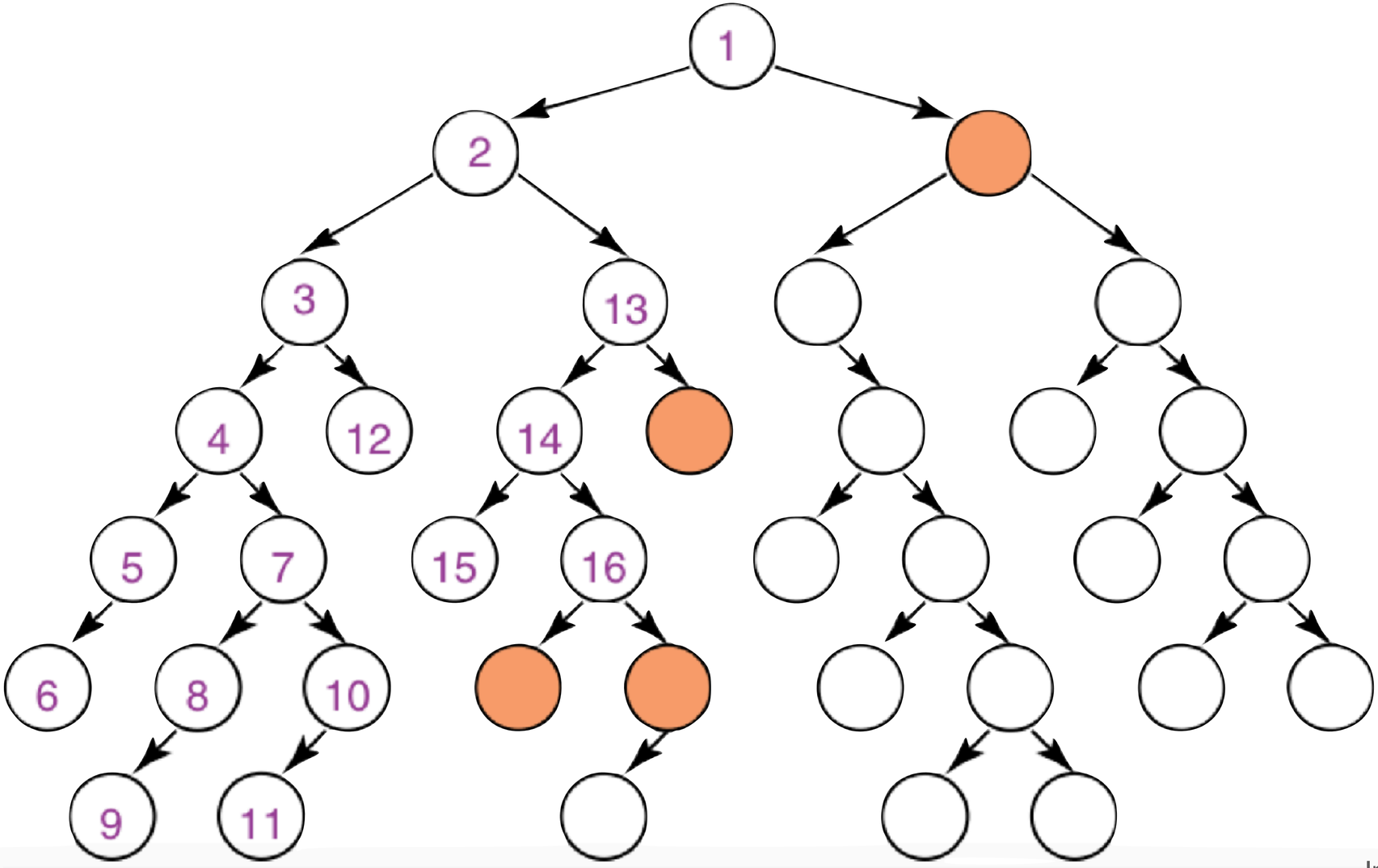


Image Credits: Peter Ljunglöf

Depth-First Search

Pseudo code

```
function Search(graph, initialState, goalState):
    initialise frontier as Stack using the initialState
    initialise exploredSet to the empty set
    while frontier is not empty:
        select and remove node from frontier
        if node.state is a goalState then return node
        add node to exploredSet
        for each child in ExpandChildNodes(node, graph):
            if child is not in frontier or exploredSet
                add child to frontier
    return failure
```

Foundations of AI

AI500

Depth-First Search

Problem Solving Performance

- **DFS is not Complete**
  - In general, DFS is not complete
  - For e.g. can enter infinite loop (enhancement can make it sort of complete)
- **DFS is not Optimal**
  - Returns the first goal found, could be anywhere in the tree, not gauranteed to be optimal
- **Time Complexity** $O(b^m)$, m = max tree depth
  - May go down long possibiliy infinite paths
  - May have to generate entire tree
- **Space Complexity**
  - Only has to store **current path** at any time
  - Use $O(bm)$ memory
  - Stores **successors** of node in the **path** (b factor)

Depth-Limited Search (DLS)

Depth-Limited Search (DLS)

How to prevent DFS from "getting lost"

- Apply a **depth limit** to the search
  - Choose a depth limit L
  - Enforce nodes at depth L have no succcessor or child
- **Solves** the infinite path problem
  - Wiil search entire tree upto depth L
- Introduces a new problem
  - **Incomplete** if solution depth $d > L$
- **Time Comlexity** $O(b^L)$
- **Space Complexity** $O(bL)$

Depth-Limited Search (DLS)

Pseudo code - Recursive

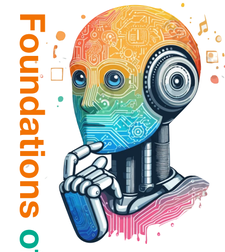
```
function DLS(node, L, goalState):
    if node.depth > L then return
    if node.state is a goalState then return node

    for each child in ExpandChildNodes(node, graph):
        DLS(child, goalState)


Call DLS(initialState, L, goalState)
```

Iterative Deepening-DFS(ID-DFS)

Iterative Deepening DFS

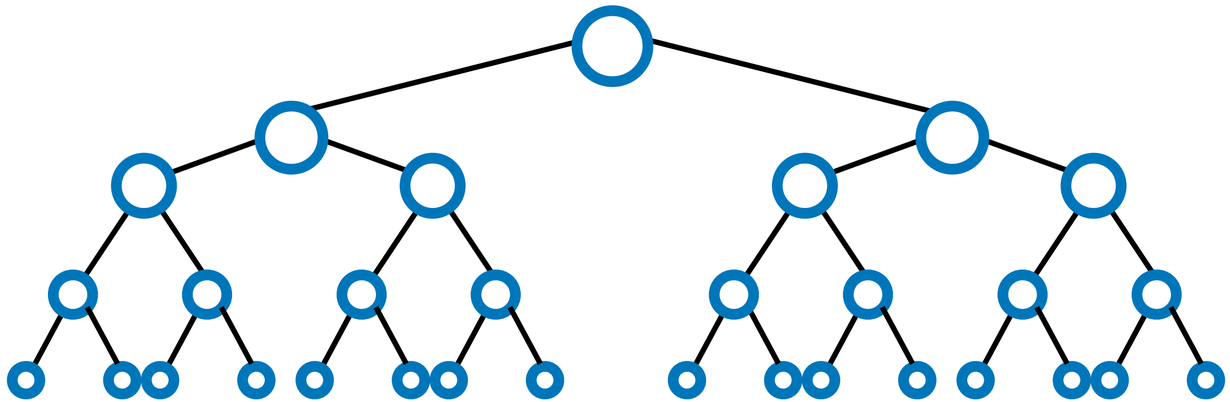


- Main Idea: Gradually increase depth limit
  - Try max depth 1, 2, 3, …. M
- Goal will be found at shallowest depth d
  - No solution found at $1 \cdots d-1$, hence $d =$ shallowest
- Completeness + Optimality of BFS
  - Gauranteed to **find** a solution
  - **Optimal** if action cost are the same
- Space Complexity of DFS $O(bd)$

ID-DFS Time Complexity

May seem wasteful, some recomputations

- Same **time** complexity as BFS
- Same **space** complexity as DFS $O(bd)$
- In practice, generates ~ twice the nodes
- In general, ID-DFS is preferable to BFS



Problem Solving Performance

- **Completeness**
  - Is it gauranteed to find a solution if it exist?
- **Optimality**
  - Does it find the optimal solution
- **Time Complexity**
  - How long does it take to find a solution
- **Space Complexity**
  - How much memory is needed to run the search?

Comparing Search Strategies

Figure 3.15 of Russel 7 Norvig 4th Ed.

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening
Complete?	Yes[1]	Yes[1,2]	No	No	Yes[1]
Optimal cost?	Yes[3]	Yes	No	No	Yes[3]
Time	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$
Space	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(bm)$	$O(b\ell)$	$O(bd)$

d: depth of the shallowest solution

b: branching factor

m: maximum depth of the tree

1: if b is finite

2: if action cost > 0

3: if all cost equal

Informed Search

Best First Search

Greedy Search

A*

Recursive Best First Search (RBFS)

(Russel & Norvig 3.5)

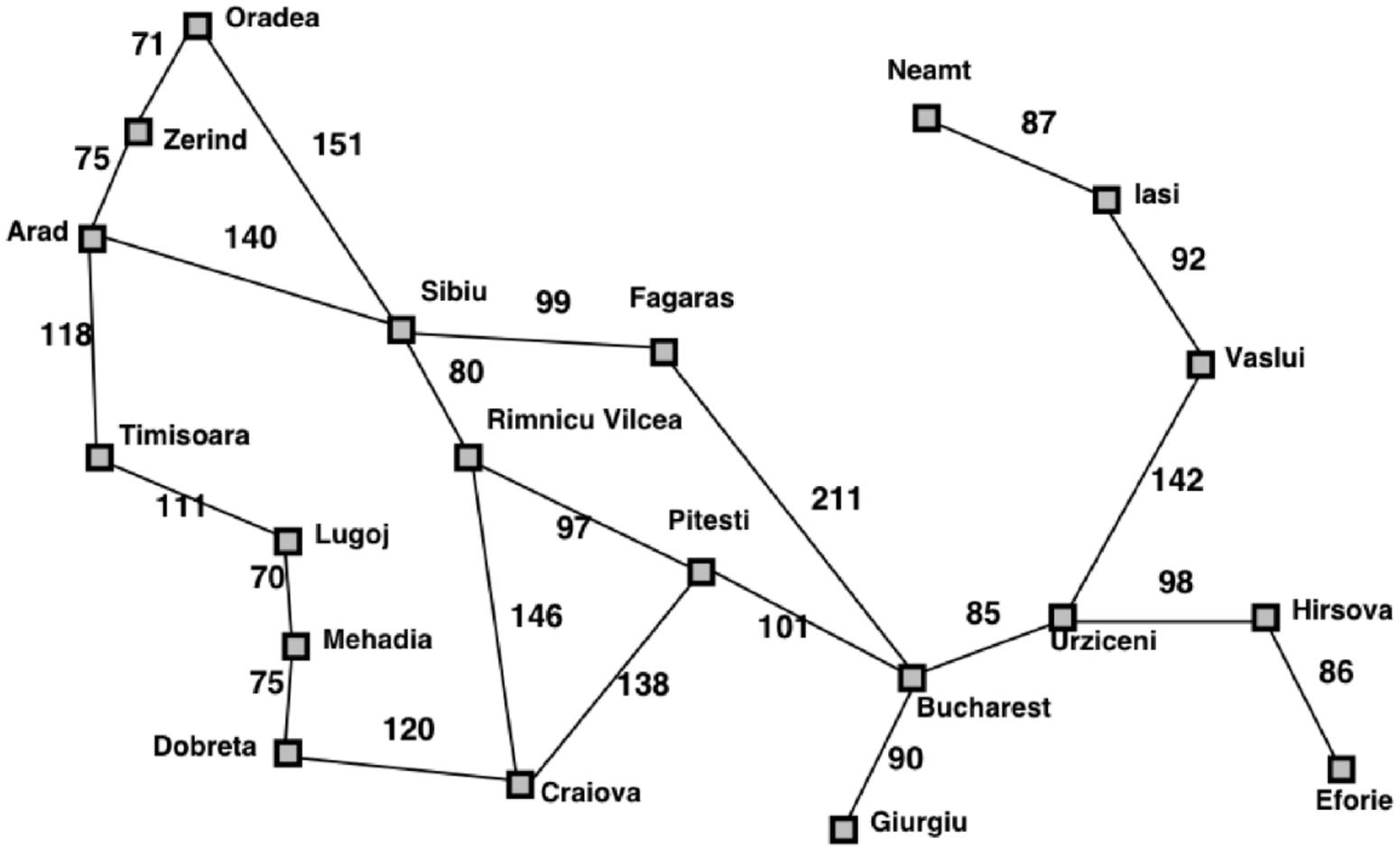
Informed (Heuristic) Search

Previous methods don't use the goal to select a path to explore.

Main idea: don't ignore the goal when selecting paths.

- Often there is extra knowledge that can guide the search: ***heuristics***.
- $h(n)$ is an estimate of the cost of the shortest path from node n to a goal node.
- $h(n)$ needs to be efficient to compute.
- $h(n)$ is an *underestimate* if there is no path from n to a goal with cost less than $h(n)$.
- An *admissible heuristic* is a nonnegative underestimating heuristic function:
  $0 \leq h(n) \leq$ *cost (best path from* n *to goal)*

Example: Romania Distances



Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Best-First Search
(general framework)

- **Definition**: A family of search algorithms that use an evaluation function $f(n)$ to decide which node to expand next.
- **Mechanism**: Maintain a *priority queue* of frontier nodes. Expand the node with the lowest $f(n)$ value.
- **Flexibility**: Different choices of $f(n)$ give different search algorithms.
  - If $f(n) = g(n)$ (path cost so far) → **Uniform Cost Search**
  - If $f(n) = h(n)$ (heuristic only) → **Greedy Best-First Search**
  - If $f(n) = h(n) + g(n)$ → **A***

Greedy Best-First Search

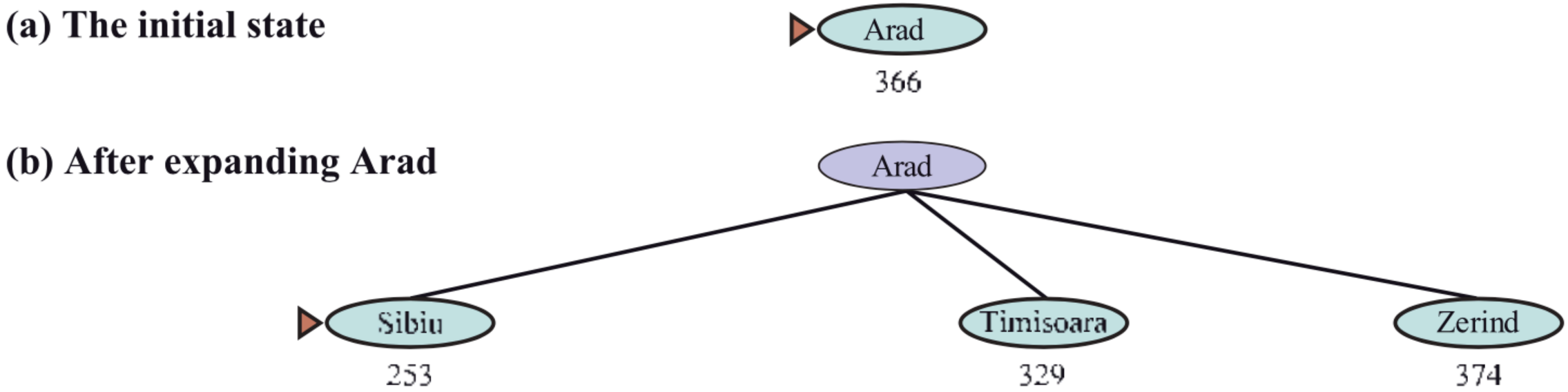
Greedy Best-First Search

3.5.1 Russel & Norvig 4th Ed.

- *Main idea*: select the path whose end is closest to a goal according to the heuristic (educated guess) function.
- Best-first search selects a path on the frontier with minimal h-value
- It treats the frontier as a priority queue ordered by h.

Greedy Best-First Search

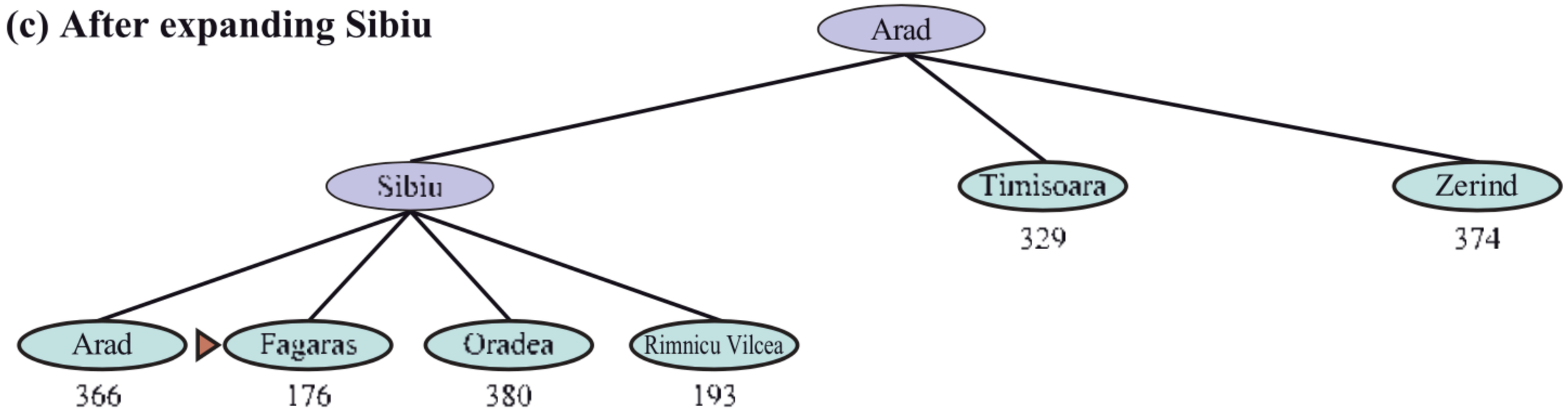
Figure 3.17 Russel & Norvig 4th Ed.



Foundations of AI
AI500

Greedy Best-First Search

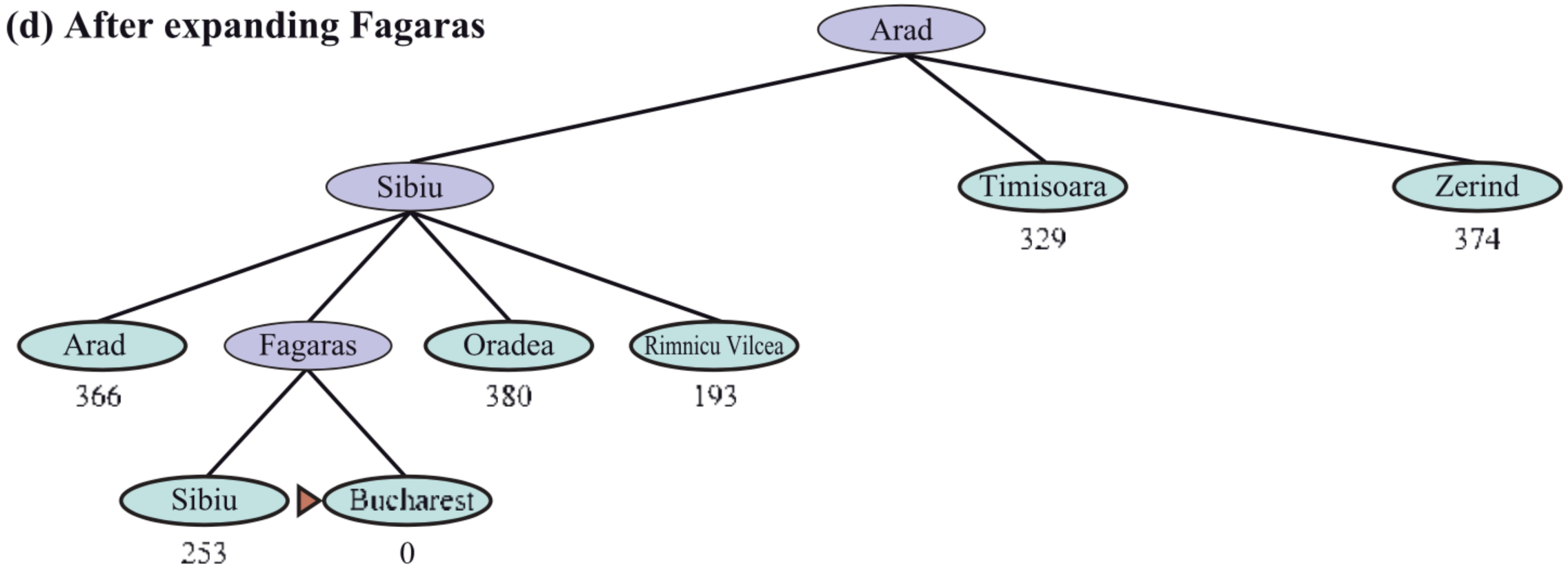
Figure 3.17 Russel & Norvig 4th Ed.



Foundations of AI
AI500

Greedy Best-First Search

Figure 3.17 Russel & Norvig 4th Ed.

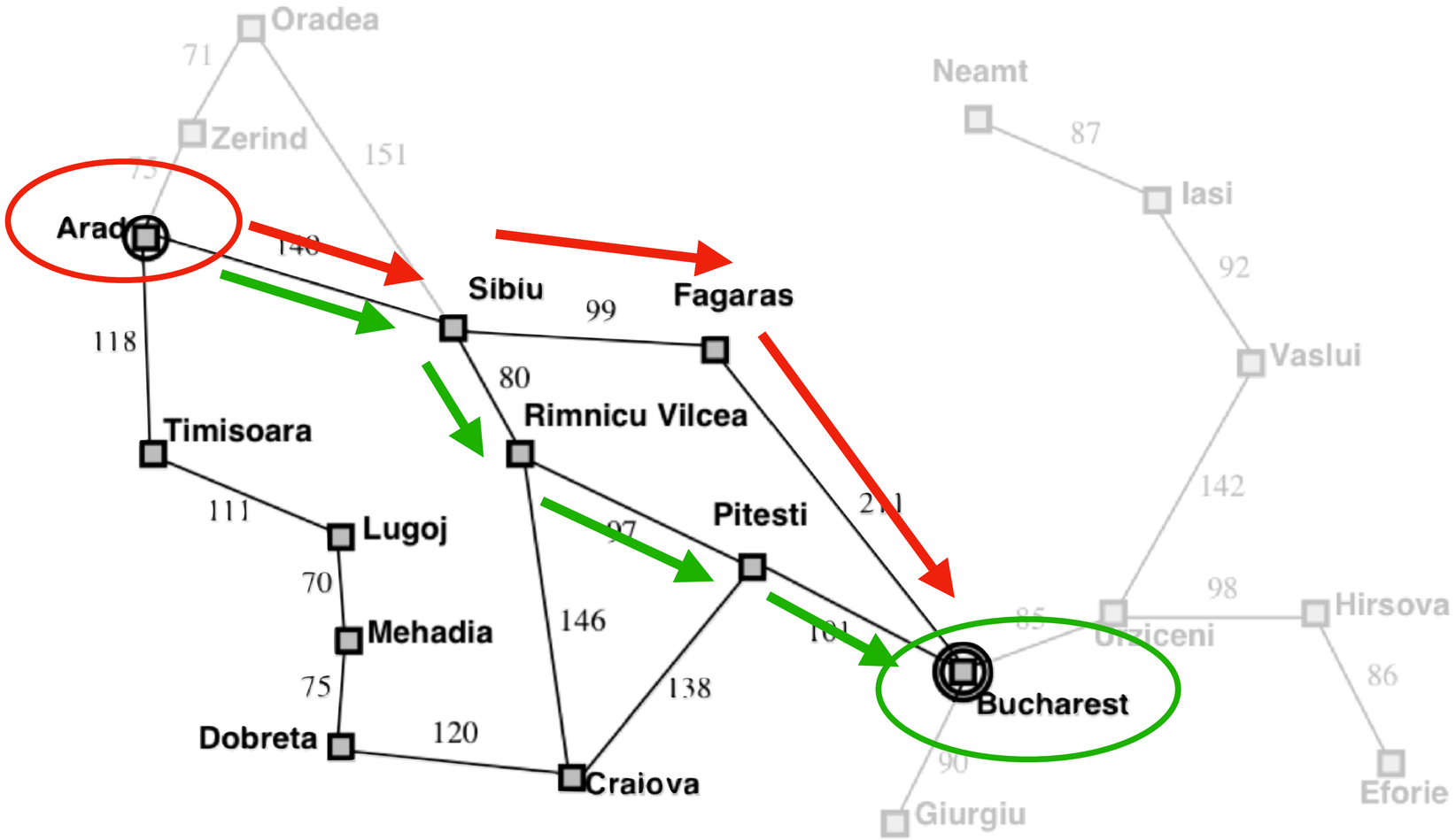


This is not the shortest path!

Greedy Best-First Search

Greedy search returns the path: *Arad–Sibiu–Fagaras–Bucharest (450km)*

The optimal path is: *Arad–Sibiu–Rimnicu–Pitesti–Bucharest (418km)*



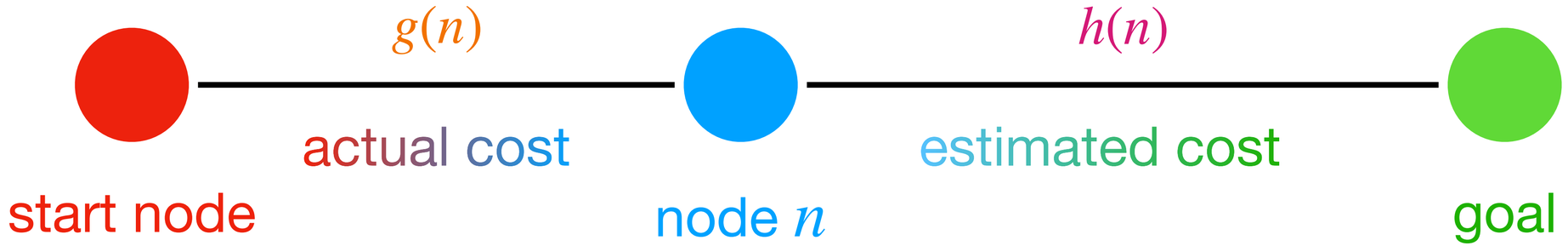
Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Foundations of AI
AI500

A* Search

A* Search

- A* search uses both path cost and heuristic values.
- $g(n)$ is the actual cost from start node to node n.
- $h(n)$ estimates the cost from the node n to a goal.
- $f(n) = g(n) + h(n)$, estimates the total path cost of going from the start node, via path to a goal:



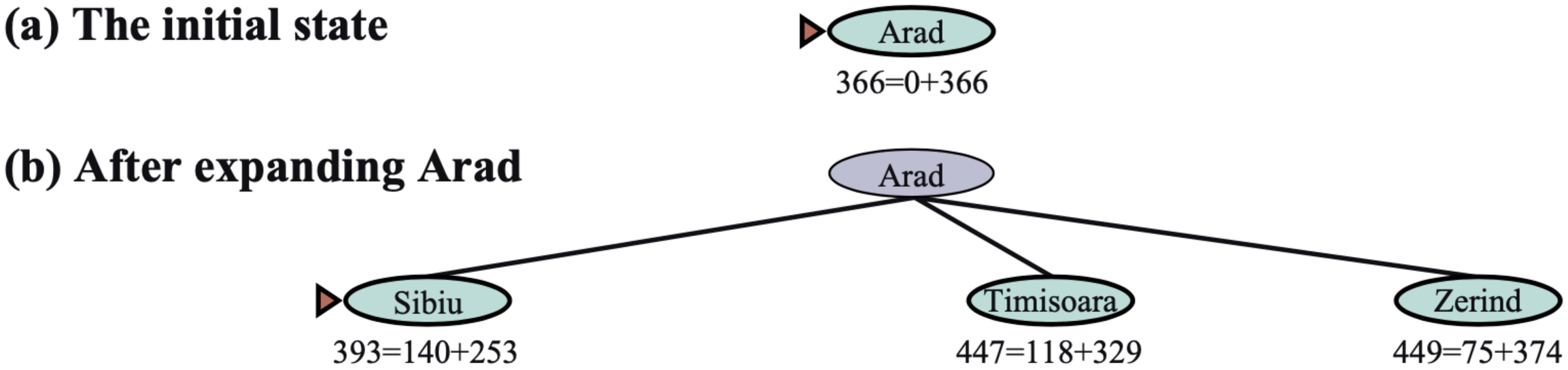
A* Search

- A* is a mix of uniform-cost search (lowest path cost) and best-first search (minimal h-value).
- It treats the frontier as a priority queue ordered by $f(n)$.
- It always selects the node on the frontier with the lowest estimated distance from the start to a goal node constrained to go via that node.

.

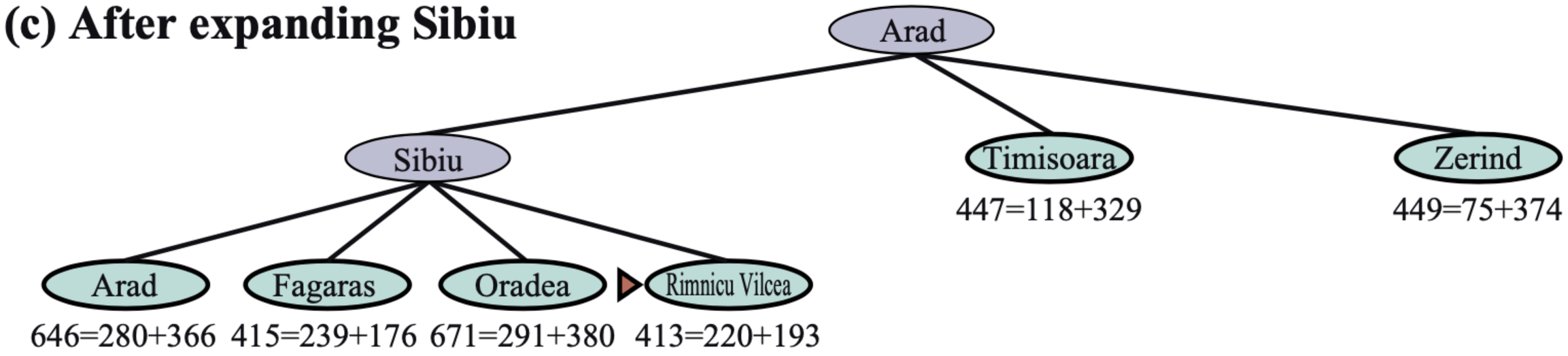
A* Search

Example: Figure 3.18 Russel & Norvig 4th Ed.



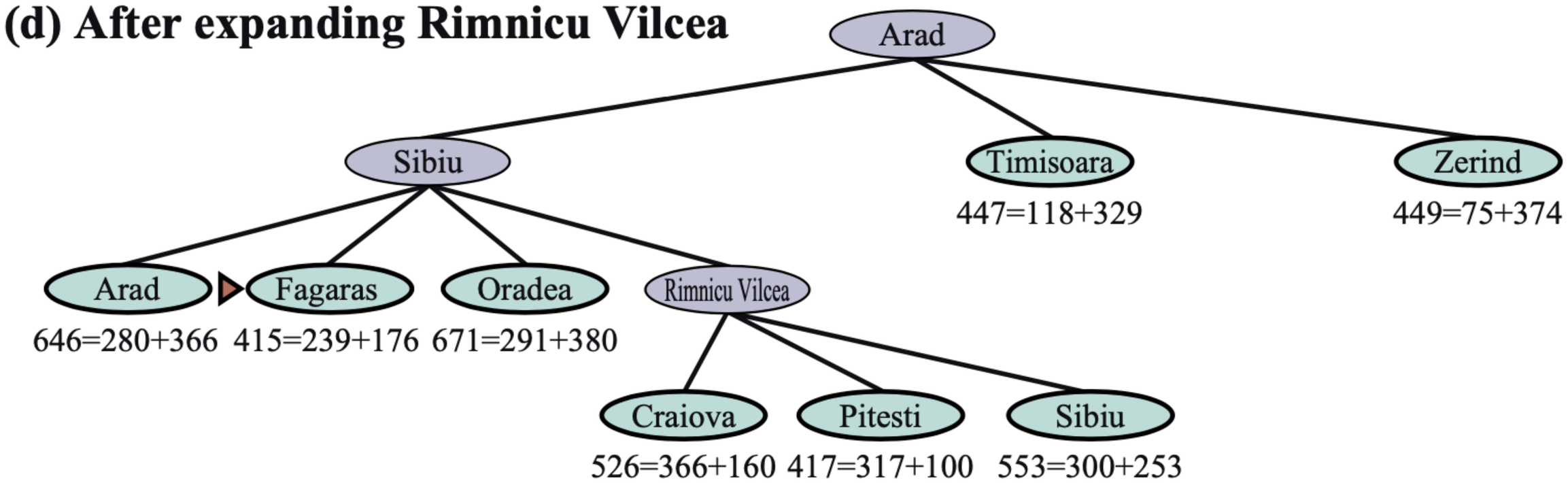
A* Search

Example: Figure 3.18 Russel & Norvig 4th Ed.



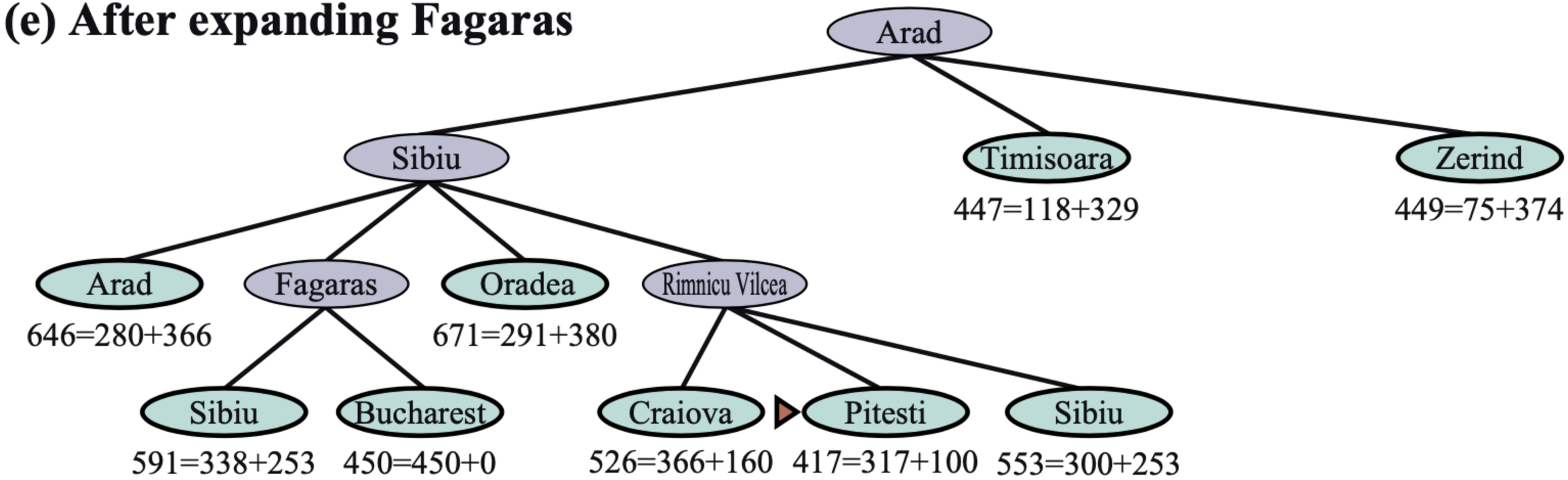
A* Search

Example: Figure 3.18 Russel & Norvig 4th Ed.



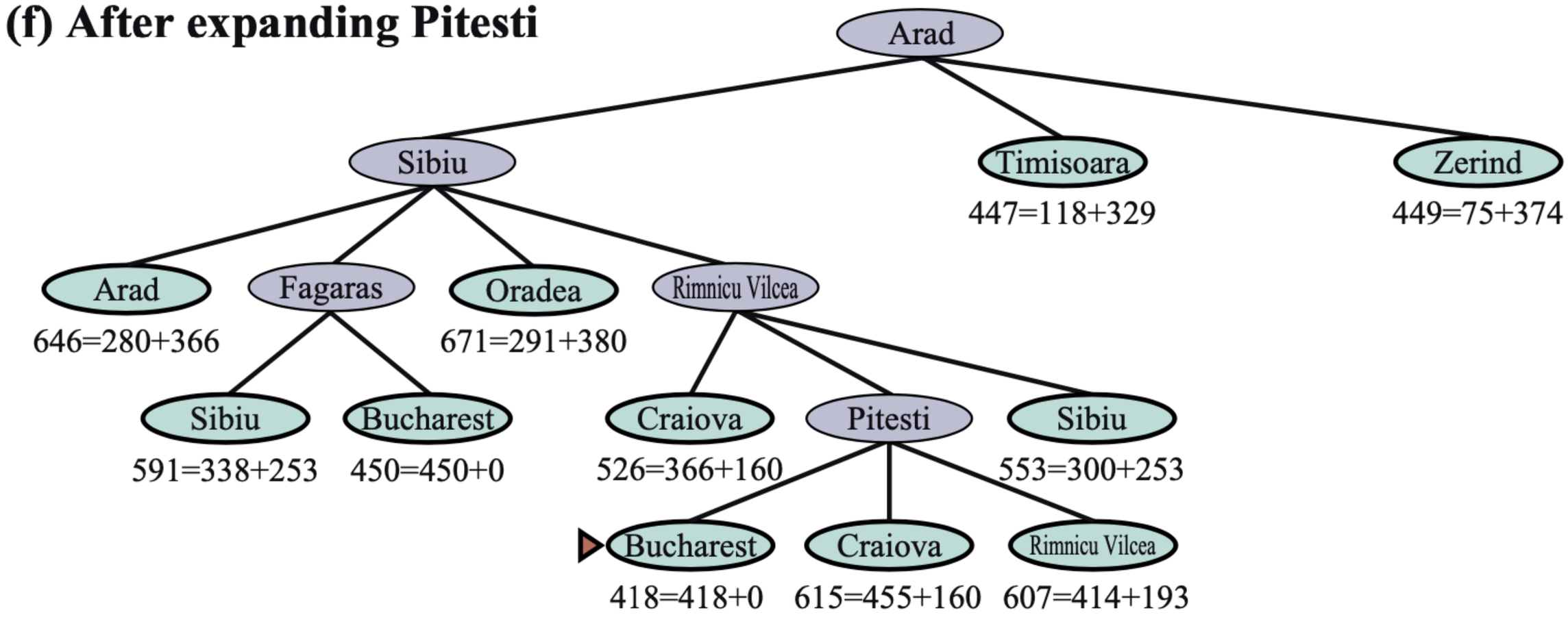
A* Search

Example: Figure 3.18 Russel & Norvig 4th Ed.



A* Search

Example: Figure 3.18 Russel & Norvig 4th Ed.



A* Tree Search

Pseudo Code

Foundations of AI

AI500

```
function AStar(h(n), goalState):
    open = PriorityQueue (f(n) = g(n) + h(n))
    open.add(initialState)

    while true:
        if open.empty then return fail
        node =  pop_mim_f(open)

        if node.state is a goalState then return node
        else open.add(ExpandChildNodes(node, goalState))
```

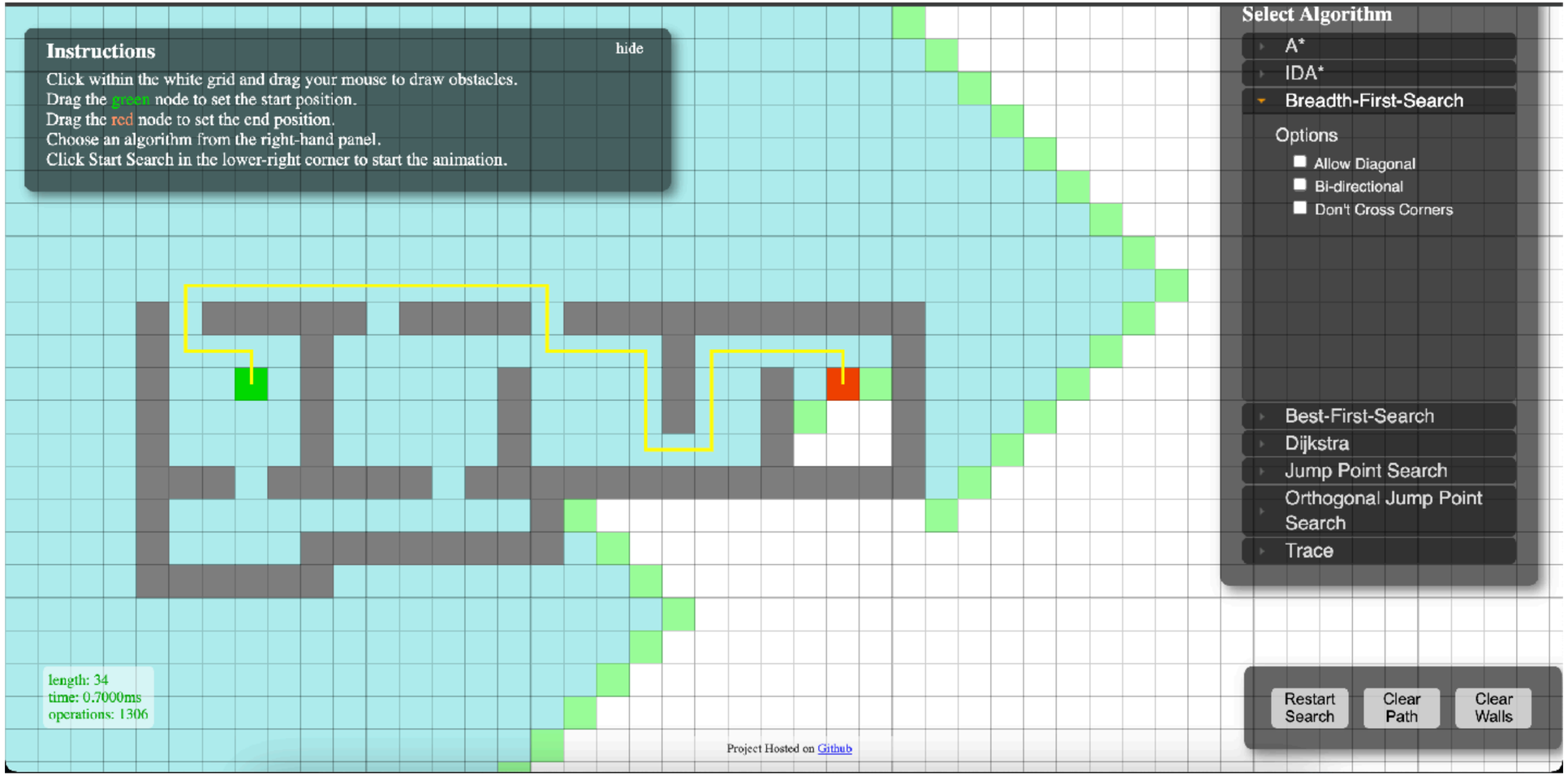
A* Performance

- A* using tree-search is **complete**
  - Will eventually search the entire tree, it will just search part where the heuristic is lower first

- A* using the Tree-Search algorithm is **optimal** if heuristic $h(n)$ is **admissible**
  - Using a heurtistic that is not admissible can produce non-optimal path

A* - Demo

https://qiao.github.io/PathFinding.js/visual/



Recursive Best First Search

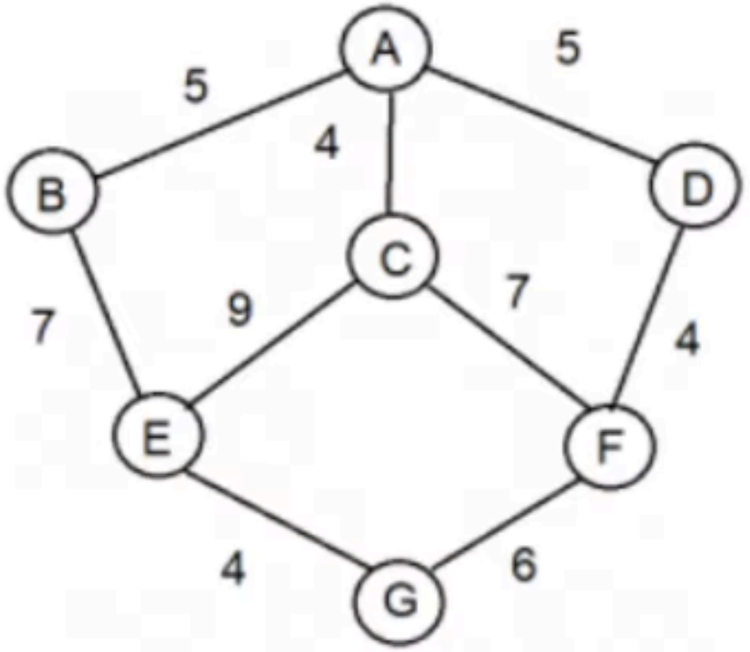
$$s.f = max(s.g + s.h, node.f)$$

$$Best = best\ successor\ node$$

$$Best.f > f_limit\ return\ failure$$

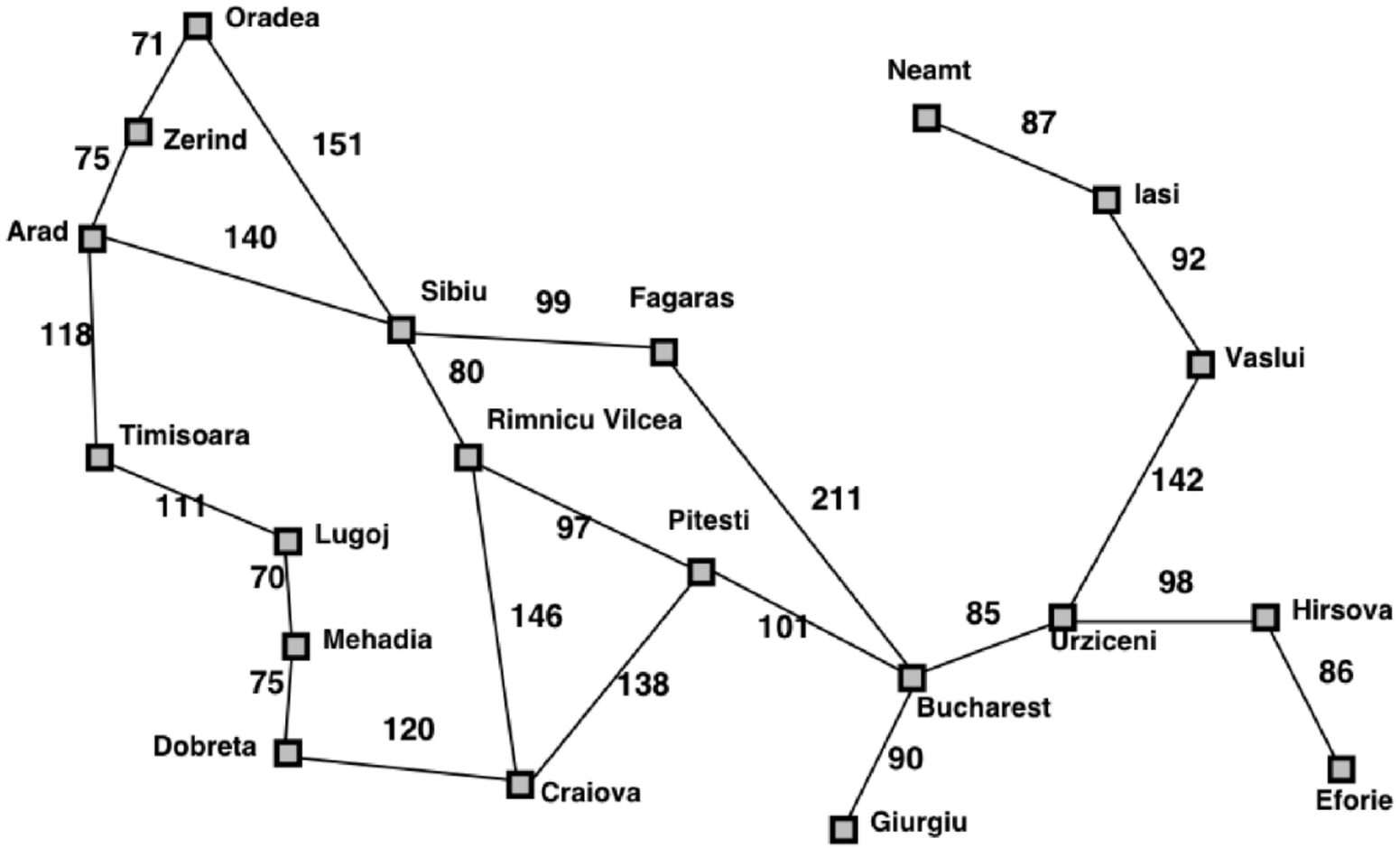
$$Alt = second\ best\ succ\ node$$

$$f_limit = min(f_limit, alt.f)$$



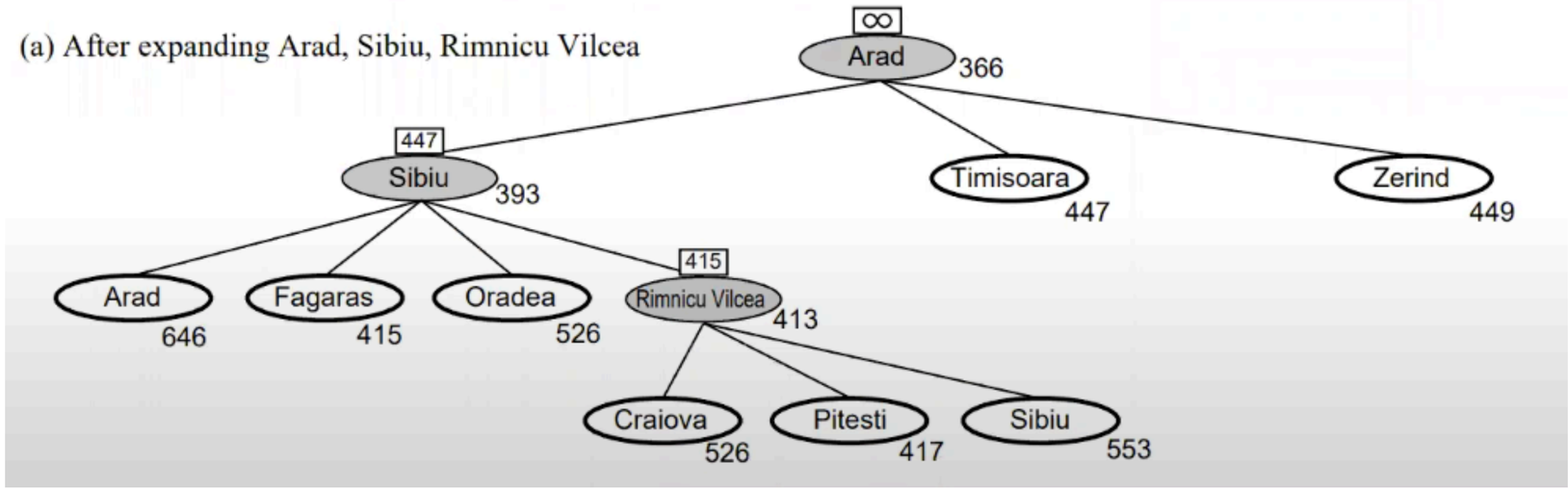
A	12
B	12
C	10
D	7
E	2
F	6
G	0

Example: Romania Distances



Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Recursive Best First Search



$$s.f = max(s.g + s.h, node.f)$$

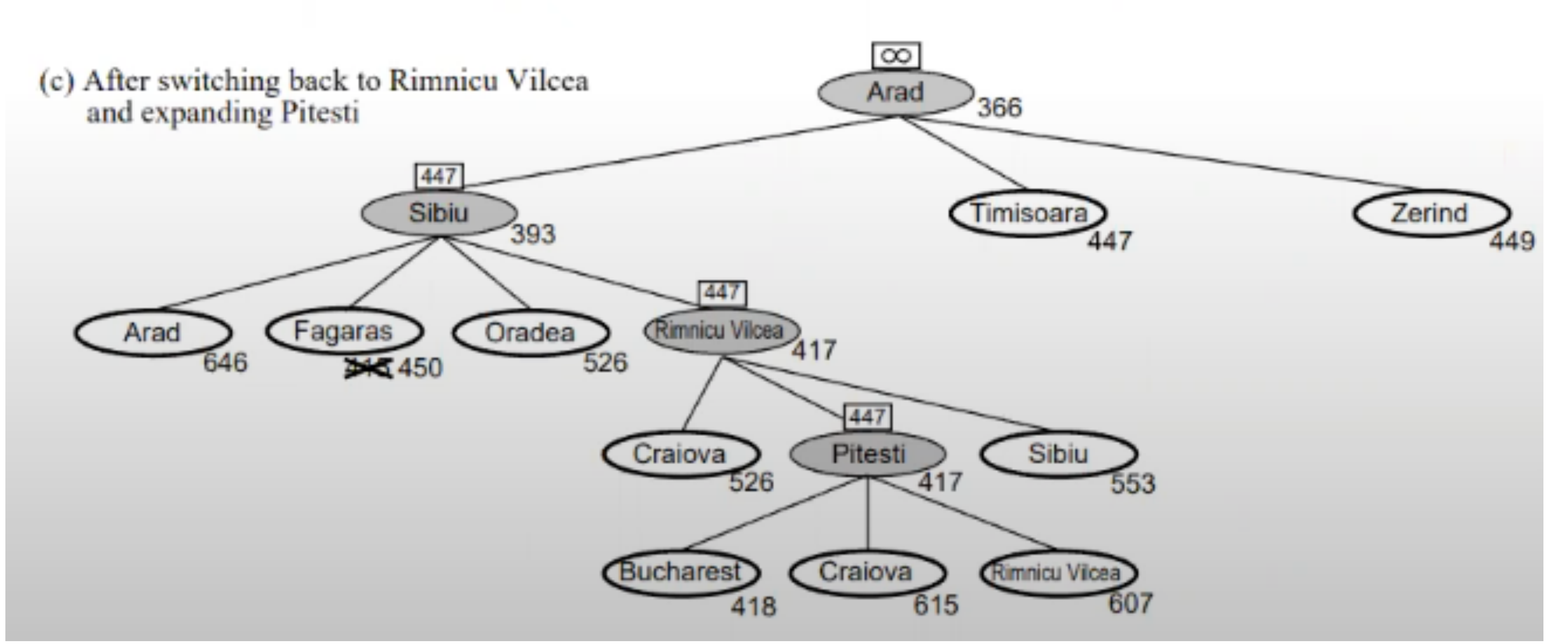
$$Best = best\ successor\ node$$

$$Best.f > f_limit\ return\ failure$$

$$Alt = second\ best\ succ\ node$$

$$f_limit = min(f_limit, alt.f)$$

Recursive Best First Search



$s.f = max(s.g + s.h, node.f)$

$Best = best\ successor\ node$

$Best.f > f_limit\ return\ failure$

$Alt = second\ best\ succ\ node$

$f_limit = min(f_limit, alt.f)$

Next …

Adversarial Search … Minimax